APEX TRADER

A Stock and Cryptocurrency Trading Database Management System with an In-Database Matching Engine and a Double-Entry Ledger

A PROJECT REPORT

submitted in partial fulfilment of the requirements for the course

BCSE302P – DATABASE SYSTEMS LAB

Submitted by

24BCE2863	Kumar Paritosh
24BCE2865	Samrender Singh Saini
24BCE2886	Vartika Chaudhary

Under the guidance of

Prof. NITHYA N S

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

October 2026

ABSTRACT

Apex Trader is a database management system for a stock and cryptocurrency trading venue. It differs from a conventional trading application in one structural respect: the order book is not held in application memory and the database is not a passive store behind it. The limit order book is a relation, order matching is a stored procedure, settlement is a journal entry, and all three execute inside a single database transaction. The database is the exchange.

The schema comprises thirty-eight base tables and thirty partitions, organised into twelve bounded-context schemas and normalised to Boyce-Codd Normal Form with three documented and trigger-verified exceptions. The matching engine implements a continuous auction with price-time priority, supporting market, limit, stop, stop-limit, trailing-stop and iceberg orders under DAY, GTC, IOC and FOK time-in-force policies. Settlement is recorded in an append-only double-entry journal in which no balance is ever stored: every balance is derived, and a DEFERRABLE INITIALLY DEFERRED constraint trigger verifies at COMMIT that each journal entry sums to zero independently for cash and for every security and currency. Value can therefore move between accounts but cannot be created or destroyed.

Risk is enforced before execution by a gate that runs inside the order transaction, so no client path can bypass it. Market-abuse surveillance is expressed as analytical SQL using self-joins and window functions. Multi-tenant isolation is enforced by PostgreSQL Row Level Security rather than by application predicates. The physical design uses declarative range partitioning, BRIN indexes on append-only time series, partial and covering indexes, and materialized views refreshed concurrently.

Correctness is demonstrated rather than asserted. The system is verified by 150 SQL assertions, 118 end-to-end API checks, 51 browser-driven interface checks, and eighteen structural invariants the database evaluates on itself. A concurrency harness drives twelve simultaneous sessions issuing three hundred orders against a single resting order of three thousand shares; the engine fills exactly three thousand, with no overfill and with zero shares or cash created or destroyed.

Keywords: relational database design, Boyce-Codd Normal Form, PL/pgSQL, limit order book, price-time priority, double-entry accounting, deferred constraints, transaction isolation, Row Level Security, declarative partitioning, PostgreSQL.

2. INTRODUCTION

2.1 Description of the project

A securities exchange performs three functions simultaneously. It maintains an order book recording every unexecuted intention to buy or sell; it matches opposing orders according to a published priority rule; and it records the resulting transfer of cash and securities so that ownership is never ambiguous. Apex Trader implements all three inside PostgreSQL.

The project supports two asset classes on a single engine. Equities and exchange-traded funds trade during a 390-minute session governed by an exchange calendar; eight cryptocurrency pairs trade continuously. The properties that differ between them (the minimum price increment, the minimum order quantity, the permitted deviation from the reference price, and the trading hours) are stored as attributes of the instrument rather than encoded as branches in the engine. A single matching procedure therefore serves both without modification.

The application tier is deliberately minimal. Order entry is one database call; the API performs no business logic, holds no cached state, and uses no object-relational mapper. Its responsibilities are authentication, setting the session identity that activates Row Level Security, converting between JSON and exact decimal types, and relaying asynchronous notifications to connected browsers.

2.2 Purpose of the database system

An exchange must answer three questions: who owns what, at what price did it change hands, and was the transaction permitted. The three are mutually dependent. The answer to the third depends on the first; the first changes as a consequence of the second. All three must remain consistent while many participants trade concurrently.

Performing these steps in application code cannot be made atomic across concurrent orders without either a global lock, which eliminates concurrency, or a distributed transaction, which introduces a second failure domain. Placing matching, settlement and risk evaluation inside one database transaction removes the problem rather than managing it: the transaction either commits in full or leaves no trace.

The second purpose is accounting integrity. In a schema that stores a balance as a column, any defect in the code that maintains it, such as a rounding error, a retried request or an interrupted loop, silently creates or destroys value, and nothing in the database contradicts the result. This

project stores no balances. Every movement of value is recorded as signed postings within a journal entry, balances are derived by aggregation, and a deferred constraint rejects any entry that does not sum to zero.

2.3 Organisation of this report

Section 3 states the objectives and the problems addressed. Section 4 specifies the software requirements and justifies the choice of database. Section 5 presents the entity-relationship model, the schema organisation and a formal normalization analysis. Section 6 documents schema creation and the sample dataset. Section 7 demonstrates database operations with executed queries and their outputs. Section 8 covers the advanced features: stored procedures, functions, triggers, views, transaction control, indexing, partitioning and access control. Section 9 presents application screenshots. Section 10 concludes and identifies future work, and Section 11 lists references.

3. OBJECTIVES

3.1 Main objectives of the project

Table 3.1 Project objectives and how each is realised

No.	Objective	Realisation in the system
1	Model a complete trading venue as a normalised relational schema	38 base tables and 30 partitions in Boyce-Codd Normal Form, organised into 12 bounded-context schemas
2	Implement order matching inside the database	engine.submit_order() in PL/pgSQL: continuous auction, price-time priority, 6 order types, 4 time-in-force policies
3	Guarantee accounting integrity by construction	Append-only double-entry journal; derived balances; deferred zero-sum constraint evaluated at COMMIT
4	Enforce risk before execution rather than after	risk.pretrade_check() invoked within the order transaction; buying power, holdings, concentration, rate, price band, daily loss
5	Detect market abuse using analytical SQL	Four rules as single queries: wash trading, layering, momentum ignition, volume anomaly; evidence stored as JSONB
6	Isolate tenants at the database layer	24 Row Level Security policies on 16 tables, driven by a per-request session variable
7	Support multiple currencies without loss of integrity	10 currencies; a conversion is a four-legged entry through an FX desk account, and the zero-sum rule applies per currency
8	Demonstrate correctness rather than assert it	150 SQL assertions, 118 API checks, 51 browser checks, 18 self-evaluated structural invariants, and a concurrency harness

3.2 Problems addressed by the system

The project originates from a three-entity design comprising USER, ACCOUNT and HOLDING, whose own documentation identified five limitations. Each is addressed in full, as set out in Table 3.2.

Table 3.2 Limitations of the original design and their resolution

Limitation in the original design	Resolution in this system
Only three entities were modelled	38 base tables across 12 bounded-context schemas, with 37 primary keys and 57 foreign keys
No separate trade-history entity	trading.execution: partitioned by time, immutable, and linked to the journal entry that settled it
Stock data was not modelled separately	reference.instrument holds tick size, lot size, price band and session rule, which the engine reads at validation time

Limitation in the original design	Resolution in this system
No views, triggers or stored procedures	12 views, 2 materialized views, 40 triggers (20 on base tables), 83 functions and 1 procedure
Limited scalability features	Declarative range partitioning with automated management, BRIN indexes on time series, partial and covering indexes, concurrent matview refresh

3.3 Scope and delimitations

The system implements a continuous auction only. Opening and closing call auctions, pro-rata allocation, and multi-venue order routing are outside scope. Corporate actions are modelled and stored but not applied to historical prices. Settlement is immediate rather than T+1, and clearing-house interposition is not modelled. These delimitations are revisited in Section 10.3.

4. SOFTWARE REQUIREMENTS

4.1 Database management system

The database is PostgreSQL 16. Version 15 is the minimum, for a reason given in Section 4.3. No feature of a commercial edition is used; the system runs on the standard open-source distribution with four extensions, all of which ship in contrib.

Table 4.1 Database platform and extensions

Component	Version / choice	Role in the system
Database engine	PostgreSQL 16 (15+ required)	Stores the schema and executes the matching engine, settlement, risk gate and surveillance rules
Server-side language	PL/pgSQL	All business logic: 84 functions and procedures across the engine, ledger, risk and reference schemas
pgcrypto	contrib extension	bcrypt password hashing performed inside the database, so no application path can persist a plaintext password
pg_trgm	contrib extension	Trigram similarity for fuzzy instrument search; supports GIN indexing of names
btree_gist	contrib extension	Enables the GiST exclusion constraint that forbids overlapping trading halts
pg_stat_statements	contrib extension	Query statistics used by the operational views in the ops schema

4.2 Programming language and tools

Table 4.2 Application technologies

Layer	Technology	Justification
API	Python 3.13, FastAPI	Asynchronous request handling; the layer is thin by design, and order entry is a single database call
Database driver	asyncpg with raw SQL	No object-relational mapper. Decimal is carried end to end; a float never reaches a NUMERIC column
Frontend	Vanilla JavaScript, no build step	Charts, depth-of-market ladder, order ticket and compliance console; updated over WebSocket
Charting	TradingView Lightweight Charts	Vendored locally so the system runs with no network access
Market data	Yahoo Finance, Finnhub, built-in simulator	A pluggable feed. The simulator makes the system demonstrable with no API key, no network and outside market hours
Verification	psql test harness, httpx, Playwright	150 SQL assertions, 118 API checks and 51 browser checks

4.3 Justification for PostgreSQL over MySQL

Three capabilities on which the design depends are unavailable or incomplete in MySQL. The choice of database is therefore not a preference but a requirement.

4.3.1 Deferrable constraint triggers

The legs of a journal entry are inserted one row at a time. Between the first insertion and the last, the entry is legitimately unbalanced. A conventional CHECK constraint, evaluated per row, would reject the first leg of every correct entry. The zero-sum rule must therefore be evaluated once per transaction, at COMMIT, which requires a DEFERRABLE INITIALLY DEFERRED constraint trigger. MySQL provides no deferred constraints, so this invariant would have to be relocated into application code, which is precisely the place this design removes it from.

4.3.2 Row Level Security

Tenant isolation is a property of the data, not of the queries that read it. PostgreSQL evaluates a policy on every access regardless of the statement issued, so a query that omits its account predicate returns an empty set rather than another trader’s rows. MySQL offers no equivalent; isolation would again depend on every query being written correctly.

4.3.3 NULLS NOT DISTINCT

A cash balance has no instrument, so ledger.balance.instrument_id is NULL for cash rows. Under standard SQL semantics two NULLs are distinct, so a conventional unique key would permit one account to hold two separate cash balances in the same currency. PostgreSQL 15 introduced NULLS NOT DISTINCT, which makes the key genuinely unique. This single clause sets the minimum version for the project.

Listing 4.1 The unique index that requires PostgreSQL 15 or later

```
CREATE UNIQUE INDEX uq_balance_slot ON ledger.balance
    (account_id, asset_class, currency, instrument_id) NULLS NOT DISTINCT;
```

4.4 Hardware and deployment requirements

The system was developed and verified on a single node. The seeded dataset of roughly 442,000 one-minute candles and 1,130 ledger postings occupies approximately 78 MB including indexes. No feature requires replication, clustering or specialised hardware; the partitioning and indexing work described in Section 8.6 is what makes vertical scale credible.

4.5 System architecture and the division of responsibility

Figure 4.1 shows where each responsibility is discharged. The arrangement is deliberately unusual for a web application: the service tier contains no business rule at all. It validates the shape of a request, issues one call to a database function, and shapes the result as JSON. Matching, settlement, risk, authorisation and audit are all below it, inside the database, where a second client (a batch job, a psql session, a future mobile application) is subject to exactly the same rules without reimplementing any of them.

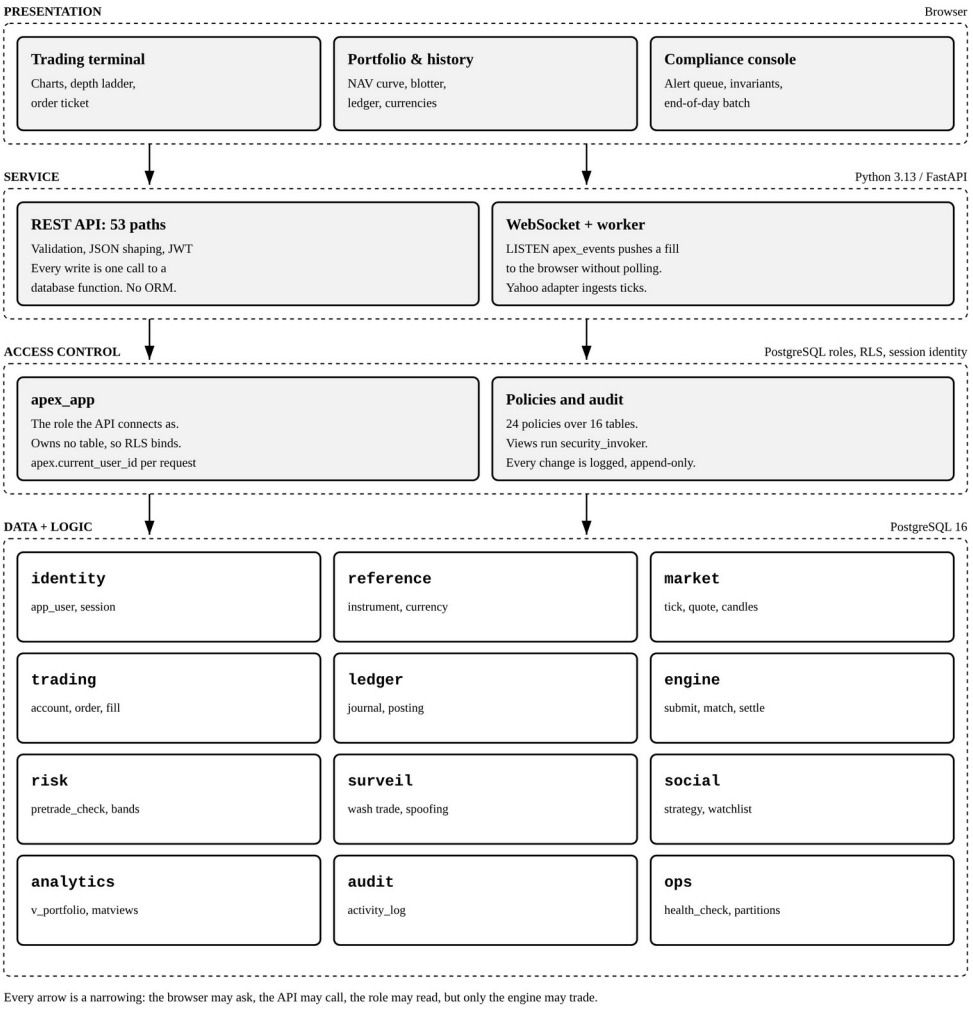


Figure 4.1 Four tiers, and the responsibilities that belong to each

This arrangement has two consequences. First, the API connects as apex_app, a role that owns no table, because a table owner is exempt from Row Level Security on its own tables and the policies would therefore not bind. Second, the browser never polls for prices: the engine issues pg_notify, a dedicated connection in the service tier holds LISTEN apex_events open, and the event is pushed to the browser over a WebSocket. The notification originates in the same transaction as the fill, so a client cannot observe a price that has not yet been committed.

5. DATABASE DESIGN

This section follows the order the guidelines set: the entity-relationship diagram first, then every core table with its attributes, primary keys and foreign keys, then the relationships between them. The schema organisation and a formal normalization analysis close the section.

5.1 Entity-relationship diagram

Figure 5.1 shows the twelve core entities: the ones a reader must follow to see how an order becomes a settled position. The complete schema has thirty-eight base tables; the exhaustive diagram is generated from the PostgreSQL system catalogue by scripts/gen_erd.py and kept in the repository as docs/er-diagram.mmd.

The diagram reads in three columns. The left column is the trading path: a user owns accounts, an account places orders, and orders produce executions. The centre column is reference and market data: an exchange lists instruments, and each instrument carries a live quote and a history of one-minute bars. The right column is the ledger: every execution is settled by one journal entry, the entry is made of postings, and the postings aggregate into balances and positions.

Each attribute is marked with its key role. PK marks a primary-key column, AK a column of an alternate (unique) key, and FK a foreign-key column. A column can carry more than one mark: trading.position.account_id is both part of the composite primary key and a foreign key to trading.account, and trading.execution has a two-column primary key because the table is partitioned by executed_at and PostgreSQL requires the partition key to be part of every unique constraint.

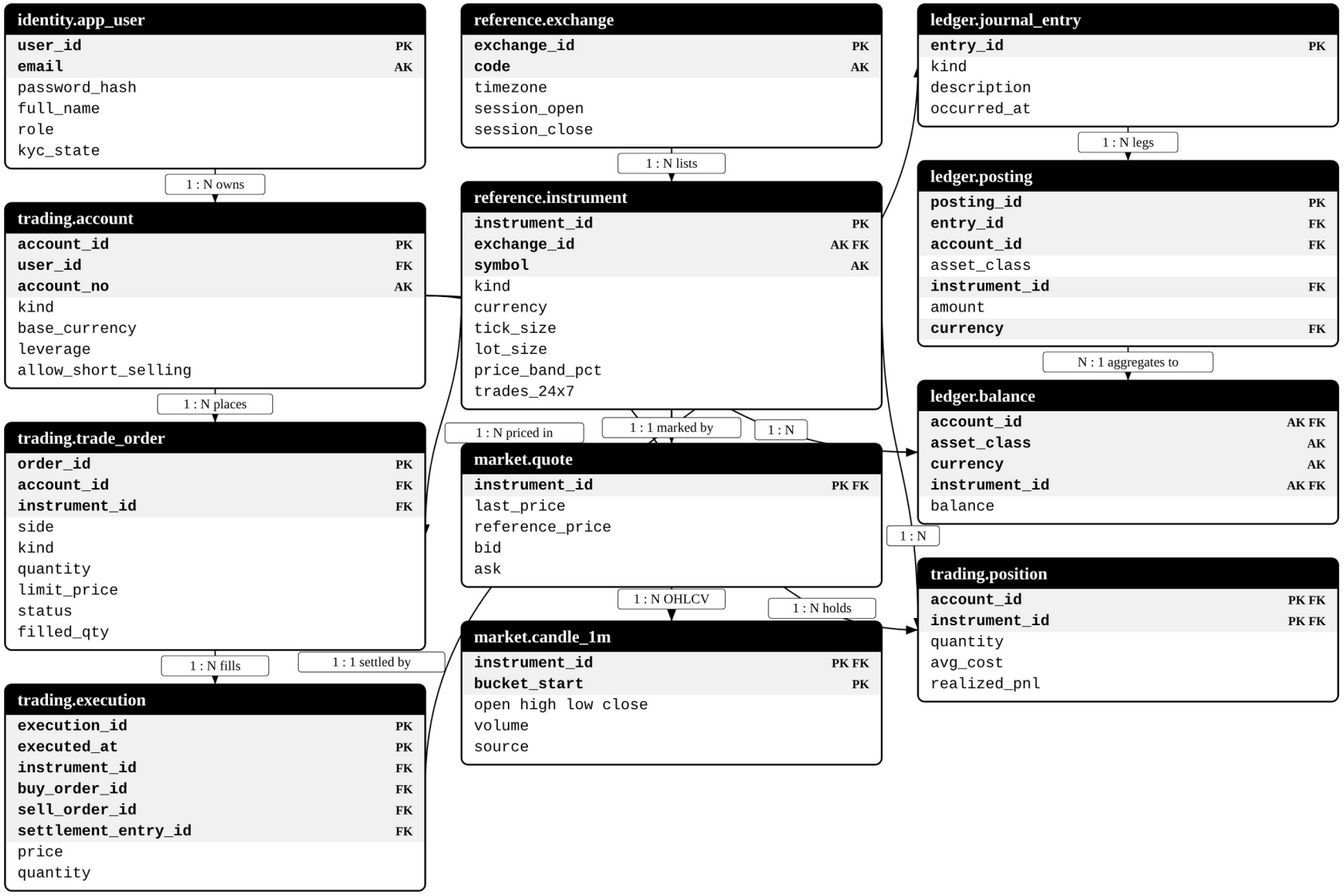


Figure 5.1 Entity-relationship diagram of the core entities. PK primary key, AK alternate (unique) key, FK foreign key.

5.2 Tables, attributes, primary keys and foreign keys

Tables 5.1 to 5.12 give every attribute of the twelve core tables, read from the system catalogue rather than from the migration files. Several tables gained columns through later migrations, so the catalogue is the only source that describes the database as it actually stands. Types such as money_amt, price_amt and currency_code are domains declared in migration 001, each carrying its own CHECK constraint; they are listed in Section 6.2.

5.2.1 identity.app_user

One row per person who can sign in. The e-mail address is unique case-insensitively, through a unique index on lower(email), so two addresses that differ only in case cannot both exist.

Table 5.1 identity.app_user

Attribute	Data type	Null	Key	References
user_id	bigint	No	PK	
public_id	uuid	No	AK	
email	email_addr	No	AK	
password_hash	text	No		
full_name	text	No		
phone	text	Yes		
date_of_birth	date	Yes		
country_code	char(2)	No		
role	identity.user_role	No		
kyc_state	identity.kyc_status	No		
kyc_verified_at	timestamptz	Yes		
totp_secret	text	Yes		
is_active	boolean	No		
failed_logins	smallint	No		
locked_until	timestamptz	Yes		
created_at	timestamptz	No		
updated_at	timestamptz	No		
last_login_at	timestamptz	Yes		

5.2.2 trading.account

A trading account belongs to one user and holds money and securities in one base currency. Five system accounts (bank, fees, float, market maker, FX) have no user and are identified instead by system_code.

Table 5.2 trading.account

Attribute	Data type	Null	Key	References
account_id	bigint	No	PK	
account_no	text	No	AK	
user_id	bigint	Yes	FK	identity.app_user
kind	trading.account_kind	No		
status	trading.account_status	No		
base_currency	currency_code	No		
leverage	numeric(5,2)	No		
maintenance_margin_pct	pct	No		
is_system	boolean	No		
system_code	text	Yes	AK	
allows_negative_cash	boolean	No		
opened_at	timestamptz	No		
closed_at	timestamptz	Yes		
allow_short_selling	boolean	No		

5.2.3 reference.exchange

A trading venue, with the time zone and session hours that decide whether its instruments are open.

Table 5.3 reference.exchange

Attribute	Data type	Null	Key	References
exchange_id	smallint	No	PK	
code	text	No	AK	
name	text	No		
country_code	char(2)	No		
timezone	text	No		
currency	currency_code	No		
session_open	time without time zone	No		
session_close	time without time zone	No		

5.2.4 reference.instrument

The security master. A symbol is unique within its exchange, not globally, so (exchange_id, symbol) is an alternate key.

Table 5.4 reference.instrument

Attribute	Data type	Null	Key	References
instrument_id	integer	No	PK	
symbol	symbol_code	No	AK	
exchange_id	smallint	No	AK FK	reference.exchange
isin	char(12)	Yes		
name	text	No		
kind	reference.instrument_kind	No		
sector	text	Yes		
industry	text	Yes		
currency	currency_code	No		
lot_size	qty_amt	No		
tick_size	price_amt	No		
price_band_pct	pct	No		
shares_outstanding	bigint	Yes		
is_tradable	boolean	No		
is_shortable	boolean	No		
listed_on	date	No		
delisted_on	date	Yes		
metadata	jsonb	No		
created_at	timestamptz	No		
trades_24x7	boolean	No		

5.2.5 trading.trade_order

An order as submitted, with its fill state. client_order_id is the idempotency key; seq_no supplies the time component of price-time priority; leaves_qty is a generated column.

Table 5.5 trading.trade_order

Attribute	Data type	Null	Key	References
order_id	bigint	No	PK	
client_order_id	uuid	No	AK	

Attribute	Data type	Null	Key	References
seq_no	bigint	No		
account_id	bigint	No	FK	trading.account
instrument_id	integer	No	FK	reference.instrument
side	trading.order_side	No		
kind	trading.order_kind	No		
tif	trading.time_in_force	No		
quantity	qty_amt	No		
limit_price	numeric(20,8)	Yes		
stop_price	numeric(20,8)	Yes		
trail_pct	pct	Yes		
peak_price	numeric(20,8)	Yes		
display_qty	numeric(20,8)	Yes		
filled_qty	qty_amt0	No		
leaves_qty	numeric(20,8)	Yes		
avg_fill_price	numeric(20,8)	Yes		
status	trading.order_status	No		
stop_armed	boolean	No		
reject_reason	text	Yes		
parent_order_id	bigint	Yes	FK	trading.trade_order
source_strategy_id	bigint	Yes	FK	social.strategy
placed_at	timestamptz	No		
updated_at	timestamptz	No		
expires_at	timestamptz	Yes		

5.2.6 trading.execution

One fill between a buy order and a sell order. Partitioned by month on executed_at, which is why the primary key includes it.

Table 5.6 trading.execution

Attribute	Data type	Null	Key	References
execution_id	bigint	No	PK	
executed_at	timestamptz	No	PK	
instrument_id	integer	No	FK	reference.instrument
price	price_amt	No		
quantity	qty_amt	No		
buy_order_id	bigint	No	FK	trading.trade_order

Attribute	Data type	Null	Key	References
sell_order_id	bigint	No	FK	trading.trade_order
buy_account_id	bigint	No	FK	trading.account
sell_account_id	bigint	No	FK	trading.account
aggressor_side	trading.order_side	No		
buy_fee	money_amt	No		
sell_fee	money_amt	No		
settlement_entry_id	bigint	Yes	FK	ledger.journal_entry

5.2.7 trading.position

The quantity of one instrument an account holds. It is a cache of the security postings and is verified against them.

Table 5.7 trading.position

Attribute	Data type	Null	Key	References
account_id	bigint	No	PK FK	trading.account
instrument_id	integer	No	PK FK	reference.instrument
quantity	money_amt	No		
avg_cost	numeric(20,8)	No		
realized_pnl	money_amt	No		
opened_at	timestamptz	No		
updated_at	timestamptz	No		

5.2.8 market.quote

The live top of book for each instrument: last price, best bid and ask, and the day statistics. One row per instrument.

Table 5.8 market.quote

Attribute	Data type	Null	Key	References
instrument_id	integer	No	PK FK	reference.instrument
bid	numeric(20,8)	Yes		
bid_size	qty_amt0	No		
ask	numeric(20,8)	Yes		
ask_size	qty_amt0	No		
last_price	numeric(20,8)	Yes		
last_size	qty_amt0	No		
day_open	numeric(20,8)	Yes		

Attribute	Data type	Null	Key	References
day_high	numeric(20,8)	Yes		
day_low	numeric(20,8)	Yes		
prev_close	numeric(20,8)	Yes		
day_volume	qty_amt0	No		
reference_price	numeric(20,8)	Yes		
updated_at	timestamptz	No		

5.2.9 market.candle_1m

One-minute OHLCV bars, partitioned by month. Every coarser chart interval is aggregated from this table.

Table 5.9 market.candle_1m

Attribute	Data type	Null	Key	References
instrument_id	integer	No	PK FK	reference.instrument
bucket_start	timestamptz	No	PK	
open	price_amt	No		
high	price_amt	No		
low	price_amt	No		
close	price_amt	No		
volume	qty_amt0	No		
trade_count	integer	No		
vwap	numeric(20,8)	Yes		
source	text	No		

5.2.10 ledger.journal_entry

The header of a double-entry transaction: what happened and when. It holds no amounts; those are in its postings.

Table 5.10 ledger.journal_entry

Attribute	Data type	Null	Key	References
entry_id	bigint	No	PK	
kind	ledger.entry_kind	No		
occurred_at	timestamptz	No		
description	text	No		
source_kind	text	Yes		

Attribute	Data type	Null	Key	References
source_id	bigint	Yes		
reverses_entry_id	bigint	Yes	FK	ledger.journal_entry
created_by	bigint	Yes	FK	identity.app_user

5.2.11 ledger.posting

One leg of a journal entry. Cash legs carry a currency and no instrument; security legs carry an instrument and no currency.

Table 5.11 ledger.posting

Attribute	Data type	Null	Key	References
posting_id	bigint	No	PK	
entry_id	bigint	No	FK	ledger.journal_entry
account_id	bigint	No	FK	trading.account
asset_class	ledger.asset_class	No		
instrument_id	integer	Yes	FK	reference.instrument
amount	money_amt	No		
memo	text	Yes		
currency	char(3)	Yes	FK	reference.currency

5.2.12 ledger.balance

The running total per account, asset class, currency and instrument. A cache of the postings; its alternate key uses NULLS NOT DISTINCT.

Table 5.12 ledger.balance

Attribute	Data type	Null	Key	References
account_id	bigint	No	AK FK	trading.account
asset_class	ledger.asset_class	No	AK	
instrument_id	integer	Yes	AK FK	reference.instrument
balance	money_amt	No		
updated_at	timestamptz	No		
currency	char(3)	Yes	AK	

5.2.13 The remaining tables

The other twenty-six base tables support the core: sessions and login attempts, currencies and exchange rates, risk limits and halts, surveillance rules and alerts, copy trading, watchlists,

analytics snapshots, the audit log and operational bookkeeping. Table 5.13 gives the primary key and foreign keys of each.

Table 5.13 Primary and foreign keys of the remaining twenty-six tables

Table	Primary key	Foreign keys
analytics.account_snapshot	account_id, as_of	account_id → trading.account
audit.activity_log	log_id	—
identity.login_attempt	attempt_id	user_id → identity.app_user
identity.user_session	session_id	user_id → identity.app_user
market.tick	ts, instrument_id, tick_id	instrument_id → reference.instrument
ops.config	key	—
ops.job_run	run_id	—
ops.schema_migration	filename	—
reference.corporate_action	action_id	instrument_id → reference.instrument
reference.currency	code	—
reference.fx_rate	base_ccy, quote_ccy, as_of	base_ccy → reference.currency; quote_ccy → reference.currency
reference.trading_session	exchange_id, session_date	exchange_id → reference.exchange
risk.alert_fire	fire_id	alert_id → risk.price_alert
risk.margin_call	call_id	account_id → trading.account
risk.price_alert	alert_id	instrument_id → reference.instrument; user_id → identity.app_user
risk.risk_limit	account_id	account_id → trading.account
risk.trading_halt	halt_id	created_by → identity.app_user; instrument_id → reference.instrument
social.copy_link	link_id	follower_order_id → trading.trade_order; strategy_id → social.strategy; subscription_id → social.copy_subscription
social.copy_queue	job_id	instrument_id → reference.instrument; strategy_id → social.strategy
social.copy_subscription	subscription_id	account_id → trading.account; strategy_id → social.strategy
social.follow	follower_user_id, strategy_id	follower_user_id → identity.app_user; strategy_id → social.strategy
social.strategy	strategy_id	owner_user_id → identity.app_user; source_account_id → trading.account
social.watchlist	watchlist_id	user_id → identity.app_user
social.watchlist_item	watchlist_id, instrument_id	instrument_id → reference.instrument; watchlist_id → social.watchlist
surveil.alert	alert_id	account_id → trading.account; instrument_id → reference.instrument; reviewed_by → identity.app_user; rule_code → surveil.rule
surveil.rule	rule_code	—

5.3 Relationships between data

Table 5.14 lists the relationships drawn in Figure 5.1 and the declaration that enforces each. Every one is a declared foreign key or primary key: none depends on the application to keep it true.

Table 5.14 Relationships between the core entities and how each is enforced

Relationship	Cardinality	Enforced by
app_user owns account	1 : N	FK account.user_id → app_user
exchange lists instrument	1 : N	FK instrument.exchange_id → exchange; AK (exchange_id, symbol)
account places trade_order	1 : N	FK trade_order.account_id → account
instrument is traded by trade_order	1 : N	FK trade_order.instrument_id → instrument
trade_order produces execution	1 : N	FK execution.buy_order_id and execution.sell_order_id → trade_order
account is party to execution	1 : N	FK execution.buy_account_id and execution.sell_account_id → account
execution settled by journal_entry	1 : 1	FK execution.settlement_entry_id → journal_entry
journal_entry contains posting	1 : N	FK posting.entry_id → journal_entry; at least two legs, checked at COMMIT
account receives posting	1 : N	FK posting.account_id → account
account holds balance	1 : N	FK balance.account_id → account; AK on the balance slot
account holds position	1 : N	PK (account_id, instrument_id); FK to account and to instrument
instrument marked by quote	1 : 1	quote.instrument_id is both PK and FK
instrument has candle_1m	1 : N	PK (instrument_id, bucket_start); FK to instrument

5.4 Schema organisation

The tables are grouped into twelve PostgreSQL schemas instead of one flat namespace. That gives readable qualified names, a separate GRANT surface for each context, and a natural place to attach security policies. It also makes the dependency direction visible: engine depends on ledger and risk, and never the reverse.

Table 5.15 The twelve schemas

Schema	Responsibility	Representative objects
identity	Users, credentials, sessions	app_user, user_session, login_attempt

Schema	Responsibility	Representative objects
reference	Security master and calendars	instrument, exchange, currency, fx_rate, trading_session
market	Market data	tick, quote, candle_1m (partitioned)
trading	Accounts, orders, fills, positions	account, trade_order, execution, position
ledger	Book of record	journal_entry, posting, balance
engine	Matching and settlement	submit_order, match_order, settle_fill, elect_stops
risk	Pre-trade control	pretrade_check, account_nav, trading_halt, price_alert
surveil	Market-abuse detection	rule, alert
social	Copy trading and watchlists	strategy, copy_subscription, watchlist
analytics	Derived read models	v_portfolio, v_blotter, account_snapshot
audit	Immutable change log	activity_log
ops	Operations	health_check, schema_migration, partition management

5.5 Normalization

The schema is in Boyce-Codd Normal Form. This section states the functional dependencies on which that claim rests, because the claim without the dependencies is an assertion rather than an analysis. Three deliberate denormalizations are documented in Section 5.5.5 together with the mechanism that keeps each provably consistent.

5.5.1 First Normal Form

A relation is in 1NF when every attribute is atomic, no repeating groups occur, and every tuple is uniquely identifiable. Thirty-seven of the thirty-eight base tables declare a primary key, and sixty-seven primary-key constraints are in force once partitions are counted. No attribute stores a delimited list and no table uses numbered columns in place of a child relation.

The thirty-eighth table, ledger.balance, has no primary key, and cannot have one. A primary key forbids NULL in every one of its columns, but this table’s natural key contains NULLs by design: instrument_id is NULL on a cash row and currency is NULL on a security row. Its tuples are instead identified by the unique index uq_balance_slot on (account_id, asset_class, currency, instrument_id) declared NULLS NOT DISTINCT, which treats two NULLs as equal and so makes the four columns a genuine candidate key. This is the clause that sets PostgreSQL 15 as the minimum version (Section 4.3.3).

Two columns need explaining. First, reference.instrument.metadata is of type JSONB. This is not a repeating group: it carries provenance that is genuinely schemaless and is never used in a predicate, a join or a decision: which provider supplied the listing, which user imported it, what exchange string the provider returned. Anything that drives behaviour is a proper column instead; trades_24x7, which decides whether an instrument is open, was added in migration 022 as a BOOLEAN column rather than a metadata key for exactly that reason. Second, surveil. alert.evidence is JSONB because the evidence for an alert differs in shape by rule: a wash trade cites the execution identifiers and the notional washed, whereas a volume anomaly cites a z-score and a rolling mean. Forcing one relation to hold the union of every rule's evidence attributes would produce a wide, mostly-null table, which is the anomaly 1NF exists to prevent rather than a remedy for it.

A third observation concerns atomicity of money. The domain money_amt is NUMERIC(28,8) and never a binary floating-point type. The value 174.73 has no exact binary representation, so a float cannot hold it exactly, and a ledger built on floats could not be proved to sum to zero.

5.5.2 Second Normal Form

A relation is in 2NF when it is in 1NF and no non-prime attribute is functionally dependent on a proper subset of any candidate key. Relations keyed by a single surrogate attribute satisfy 2NF trivially. The relations with composite keys are examined individually in Table 5.16.

Table 5.16 Composite-key relations and their functional dependencies

Relation	Candidate key	Functional dependency	In 2NF
trading.position	(account_id, instrument_id)	{account_id, instrument_id} → quantity, avg_cost, realized_pnl	Yes
ledger.balance	(account_id, asset_class, currency, instrument_id)	{all four} → balance, updated_at	Yes
reference.fx_rate	(base_ccy, quote_ccy, as_of)	{base, quote, as_of} → rate, source	Yes
reference.trading_session	(exchange_id, session_date)	{exchange_id, session_date} → is_trading_day, open_ts, close_ts	Yes
social.watchlist_item	(watchlist_id, instrument_id)	{watchlist_id, instrument_id} → position, note, added_at	Yes

The violation this design must avoid is the inclusion of instrument.symbol or account.account_no on trading.position for convenience of display. The symbol is determined by instrument_id alone, which is a proper subset of the key, and its inclusion would be a partial dependency. It is

therefore absent, and every view that displays a symbol obtains it by joining reference. instrument.

reference.fx_rate shows why the key matters. Keying an exchange rate on the currency pair alone would satisfy 2NF and would be factually wrong: a rate is a time series, and a trade settled three weeks ago must be converted at that date's rate. Omitting as_of from the key would apply a look-ahead rate to historical records. Correct normalization and correct accounting coincide here.

5.5.3 Third Normal Form

A relation is in 3NF when it is in 2NF and no non-prime attribute is transitively dependent on a candidate key. The transitive dependencies a trading schema invites are enumerated in Table 5. 17 together with the derived object that holds each value instead.

Table 5.17 Transitive dependencies avoided, and the derived objects used instead

Attribute not stored	Transitive chain	Where the value is obtained
trade_order.symbol	order → instrument → symbol	Join to reference.instrument
trade_order.tick_size	order → instrument → tick_size	Read by engine.validate_order()
execution.notional	quantity × price, both present	Computed in analytics.v_blotter
position.market_value	position → instrument → quote → last_price	Computed in analytics.v_portfolio
account.nav	account → positions → quotes	Function risk.account_nav()
app_user.account_count	user → accounts → count	Aggregate, never stored

The governing rule applied throughout the schema is that a value which can be derived is expressed as a view or a function and not as a column. The fourteen views and eighty-four routines exist largely to hold values that would otherwise have become stale attributes.

5.5.4 Boyce-Codd Normal Form

A relation is in BCNF when, for every non-trivial functional dependency X → Y, X is a superkey. Only relations with more than one candidate key can violate BCNF while satisfying 3NF, so those relations are enumerated in Table 5.18.

Table 5.18 Relations with multiple candidate keys

Relation	Candidate keys	Assessment
reference.instrument	instrument_id; (exchange_id, symbol)	The two keys share no attribute. All non-prime attributes depend on the full instrument identity. isin is NOT a candidate key: it is nullable and carries no unique constraint, since no cryptocurrency pair has one
identity.app_user	user_id; public_id; lower(email)	Three keys, the third enforced by a unique expression index so that two addresses differing only in case cannot coexist. Each determines the whole tuple
trading.account	account_id; account_no; system_code	system_code is unique and nullable; only the five system accounts carry one
reference.currency	code	A single candidate key. name and symbol are neither unique nor determining; two currencies may share the symbol used for the dollar

No relation contains an attribute determined by a non-superkey, so the schema is in Boyce-Codd Normal Form and no decomposition beyond 3NF was required. The canonical BCNF counterexample requires two overlapping composite candidate keys; no relation here has such a pair.

The consequence of ignoring BCNF here is concrete. Had symbol alone been declared unique, the venue could never list the same ticker on two exchanges, and the remedy would later have been a decomposition of a populated table rather than a change to a constraint.

5.5.5 Deliberate denormalizations

Three relations store values derivable from others. Each is a performance cache, each is maintained by trigger rather than by application code, and each is verifiable against its source. A redundancy that cannot be checked is a defect awaiting discovery; a redundancy with a proof is an engineering decision.

Table 5.19 Denormalizations, their maintenance and their verification

Relation	Derivable from	Maintained by	Verified by
ledger.balance	SUM over ledger.posting	trg_posting_updates_balance	ledger.verify_integrity(), which reports drifted slots
trading.position	Security postings per instrument	trg_posting_syncs_position	ops.health_check(): positions_match_ledger
market.quote	Latest market.tick plus session aggregates	engine.settle_fill()	ops.health_check(): bands_contain_their_books

A fourth case is instructive by contrast. market.candle_1m is an aggregate of market.tick and is stored; the five-minute, hourly and daily intervals are not stored at all but derived by market. candles() from the minute table. The boundary between "cache this" and "derive this" is drawn at the base resolution, which guarantees that an hourly bar can never contradict the sixty minute bars within it.

5.5.6 What normalization does not provide

Normal forms eliminate redundancy within a relation. They say nothing about conservation across relations, which is the property a ledger actually requires: the postings of a journal entry must sum to zero, independently for cash and for every security and currency. No normal form expresses this constraint. It requires a deferred constraint trigger evaluated at COMMIT, as described in Section 8.3. A schema may be in Boyce-Codd Normal Form and still permit the creation of money.

6. DATABASE CREATION AND SAMPLE DATA

6.1 Migration-controlled schema creation

The schema is not created by a single script. It is built by twenty-eight ordered migration files held under db/migrations, each applied exactly once and recorded in ops.schema_migration together with the SHA-256 checksum of the file that was applied. Re-running the migration driver is therefore a no-operation, and an edited file that has already been applied is detected rather than silently skipped. This is the mechanism by which the examiner, the developers and the demonstration machine are guaranteed to be looking at the same database.

Listing 6.1 Migrations are applied in order and recorded with a checksum

```
$ ./scripts/db.sh migrate
  001_foundation.sql          applied
  002_identity.sql            applied
  ...
  028_mark_divergence.sql     applied

SELECT version, applied_at, left(checksum, 12) AS checksum
  FROM ops.schema_migration ORDER BY version;
```

6.2 Foundation objects: extensions, schemas, domains and enumerated types

Before any table exists, migration 001 sets up the vocabulary the rest of the schema is written in: four extensions, twelve schemas, nine domains and the enumerated types. A domain is a type that carries its own CHECK, so a rule such as "a price is positive" is written once and enforced on every column of that type, instead of being repeated, and eventually forgotten, column by column.

Listing 6.2 Extensions, schemas, domains and enumerated types

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;            -- password hashing, gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS btree_gist;          -- GiST over scalars, for EXCLUDE
CREATE EXTENSION IF NOT EXISTS pg_trgm;             -- trigram index for symbol search
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;  -- query performance telemetry

CREATE SCHEMA identity;   CREATE SCHEMA reference;  CREATE SCHEMA market;
CREATE SCHEMA trading;    CREATE SCHEMA ledger;     CREATE SCHEMA engine;
CREATE SCHEMA risk;       CREATE SCHEMA surveil;    CREATE SCHEMA social;
CREATE SCHEMA analytics;  CREATE SCHEMA audit;      CREATE SCHEMA ops;

CREATE DOMAIN money_amt     AS NUMERIC(28,8);
CREATE DOMAIN price_amt     AS NUMERIC(20,8) CHECK (VALUE > 0);
CREATE DOMAIN qty_amt       AS NUMERIC(20,8) CHECK (VALUE > 0);
CREATE DOMAIN qty_amt0      AS NUMERIC(20,8) CHECK (VALUE >= 0);
CREATE DOMAIN pct           AS NUMERIC(9,4)  CHECK (VALUE >= 0 AND VALUE <= 100);
CREATE DOMAIN bps           AS NUMERIC(9,4)  CHECK (VALUE >= 0 AND VALUE <= 100000);
CREATE DOMAIN currency_code AS CHAR(3)       CHECK (VALUE ~ '^[A-Z]{3}$');
CREATE DOMAIN symbol_code   AS TEXT          CHECK (VALUE ~
    '^[A-Z0-9][A-Z0-9._-]{0,19}$');
CREATE DOMAIN email_addr    AS TEXT
       CHECK (VALUE ~ '^[^@[:space:]]+@[^@[:space:]]+\.[A-Za-z]{2,}$');

CREATE TYPE trading.order_side    AS ENUM ('buy', 'sell');
CREATE TYPE trading.order_kind    AS ENUM ('market', 'limit', 'stop', 'stop_limit',
                                           'trailing_stop', 'iceberg');
```

```
CREATE TYPE trading.time_in_force AS ENUM ('day', 'gtc', 'ioc', 'fok');
CREATE TYPE trading.order_status  AS ENUM ('pending_new', 'rejected', 'working',
                                           'partially_filled', 'filled',
                                           'cancelled', 'expired');
CREATE TYPE ledger.asset_class    AS ENUM ('cash', 'security');
```

Every monetary and quantity column is NUMERIC, never a floating-point type. Floating-point addition is not exact, and a ledger whose amounts cannot be added exactly cannot be proved to sum to zero. The deferred constraint of Section 8.3 depends on this choice.

6.3 CREATE TABLE statements

The twelve core tables follow. They are not copied from the migration files: several tables gained columns and constraints in later migrations, so each statement below was reconstructed from pg_attribute and pg_constraint and shows the table as the database holds it today. Every constraint keeps the name it was declared with, which is also the name PostgreSQL prints when the constraint rejects a row.

6.3.1 Identity and reference data

identity.app_user stores no password, only a bcrypt hash. The e-mail address is unique without regard to case, through a unique index on lower(email) rather than a plain UNIQUE constraint, and kyc_timestamp_consistent ties the KYC state to its timestamp so a user cannot be marked verified without a verification date.

Listing 6.3 identity.app_user

```
CREATE TABLE identity.app_user (
    user_id          BIGINT GENERATED ALWAYS AS IDENTITY,
    public_id        UUID NOT NULL DEFAULT gen_random_uuid(),
    email            email_addr NOT NULL,
    password_hash    TEXT NOT NULL,
    full_name        TEXT NOT NULL,
    phone            TEXT,
    date_of_birth    DATE,
    country_code     CHAR(2) NOT NULL DEFAULT 'IN',
    role             identity.user_role NOT NULL DEFAULT 'trader',
    kyc_state        identity.kyc_status NOT NULL DEFAULT 'unverified',
    kyc_verified_at  TIMESTAMPTZ,
    totp_secret      TEXT,
    is_active        BOOLEAN NOT NULL DEFAULT true,
    failed_logins    SMALLINT NOT NULL DEFAULT 0,
    locked_until     TIMESTAMPTZ,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_login_at    TIMESTAMPTZ,
    CONSTRAINT app_user_pkey
        PRIMARY KEY (user_id),
    CONSTRAINT app_user_public_id_key
        UNIQUE (public_id),
    CONSTRAINT app_user_country_code_check
        CHECK (country_code ~ '^[A-Z]{2}$'),
    CONSTRAINT app_user_date_of_birth_check
        CHECK (date_of_birth IS NULL OR date_of_birth < (CURRENT_DATE - '18 years')),
    CONSTRAINT app_user_failed_logins_check
        CHECK (failed_logins >= 0),
    CONSTRAINT app_user_full_name_check
        CHECK (length(btrim(full_name)) >= 2 AND length(btrim(full_name)) <= 120),
    CONSTRAINT app_user_phone_check
        CHECK (phone IS NULL OR phone ~ '^\+?[0-9 ()-]{7,20}$'),
    CONSTRAINT kyc_timestamp_consistent
```

```
        CHECK ((kyc_state = 'verified') = (kyc_verified_at IS NOT NULL))
);

CREATE UNIQUE INDEX uq_app_user_email_ci
    ON identity.app_user (lower(email));
```

reference.exchange carries its own sanity checks: tz_is_real rejects a time zone name the server does not recognise, and session_ordered rejects a session that closes before it opens. In reference.instrument a symbol is unique per exchange, not globally, which is the composite key symbol_unique_per_exchange.

Listing 6.4 reference.exchange

```
CREATE TABLE reference.exchange (
    exchange_id    SMALLINT GENERATED ALWAYS AS IDENTITY,
    code           TEXT NOT NULL,
    name           TEXT NOT NULL,
    country_code   CHAR(2) NOT NULL,
    timezone       TEXT NOT NULL,
    currency       currency_code NOT NULL,
    session_open   time without time zone NOT NULL,
    session_close  time without time zone NOT NULL,
    CONSTRAINT exchange_pkey
        PRIMARY KEY (exchange_id),
    CONSTRAINT exchange_code_key
        UNIQUE (code),
    CONSTRAINT exchange_code_check
        CHECK (code ~ '^[A-Z]{2,12}$'),
    CONSTRAINT exchange_country_code_check
        CHECK (country_code ~ '^[A-Z]{2}$'),
    CONSTRAINT session_ordered
        CHECK (session_close > session_open),
    CONSTRAINT tz_is_real
        CHECK ((now() AT TIME ZONE timezone) IS NOT NULL)
);
```

Listing 6.5 reference.instrument

```
CREATE TABLE reference.instrument (
    instrument_id       INTEGER GENERATED ALWAYS AS IDENTITY,
    symbol              symbol_code NOT NULL,
    exchange_id         SMALLINT NOT NULL,
    isin                CHAR(12),
    name                TEXT NOT NULL,
    kind                reference.instrument_kind NOT NULL DEFAULT 'equity',
    sector              TEXT,
    industry            TEXT,
    currency            currency_code NOT NULL DEFAULT 'USD',
    lot_size            qty_amt NOT NULL DEFAULT 1,
    tick_size           price_amt NOT NULL DEFAULT 0.01,
    price_band_pct      pct NOT NULL DEFAULT 20,
    shares_outstanding  BIGINT,
    is_tradable         BOOLEAN NOT NULL DEFAULT true,
    is_shortable        BOOLEAN NOT NULL DEFAULT false,
    listed_on           DATE NOT NULL DEFAULT CURRENT_DATE,
    delisted_on         DATE,
    metadata            JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    trades_24x7         BOOLEAN NOT NULL DEFAULT false,
    CONSTRAINT instrument_pkey
        PRIMARY KEY (instrument_id),
    CONSTRAINT symbol_unique_per_exchange
        UNIQUE (exchange_id, symbol),
    CONSTRAINT instrument_exchange_id_fkey
        FOREIGN KEY (exchange_id) REFERENCES reference.exchange(exchange_id),
    CONSTRAINT delisted_not_tradable
        CHECK (delisted_on IS NULL OR NOT is_tradable),
    CONSTRAINT instrument_isin_check
        CHECK (isin IS NULL OR isin ~ '^[A-Z]{2}[A-Z0-9]{9}[0-9]$'),
    CONSTRAINT instrument_shares_outstanding_check
        CHECK (shares_outstanding IS NULL OR shares_outstanding > 0),
    CONSTRAINT listing_window_ordered
        CHECK (delisted_on IS NULL OR delisted_on >= listed_on)
);
```

6.3.2 Accounts, orders, executions and positions

trading.account encodes two business rules as CHECK constraints. system_account_shape says an account is either a system account, with a system_code and no user, or a customer account, with a user and no system_code, and never a mix. short_requires_margin forbids short selling on a cash account.

Listing 6.6 trading.account

```
CREATE TABLE trading.account (
    account_id              BIGINT GENERATED ALWAYS AS IDENTITY,
    account_no              TEXT NOT NULL,
    user_id                 BIGINT,
    kind                    trading.account_kind NOT NULL DEFAULT 'cash',
    status                  trading.account_status NOT NULL DEFAULT 'active',
    base_currency           currency_code NOT NULL DEFAULT 'USD',
    leverage                NUMERIC(5,2) NOT NULL DEFAULT 1.00,
    maintenance_margin_pct  pct NOT NULL DEFAULT 25,
    is_system               BOOLEAN NOT NULL DEFAULT false,
    system_code             TEXT,
    allows_negative_cash    BOOLEAN NOT NULL DEFAULT false,
    opened_at               TIMESTAMPTZ NOT NULL DEFAULT now(),
    closed_at               TIMESTAMPTZ,
    allow_short_selling     BOOLEAN NOT NULL DEFAULT false,
    CONSTRAINT account_pkey
        PRIMARY KEY (account_id),
    CONSTRAINT account_account_no_key
        UNIQUE (account_no),
    CONSTRAINT account_system_code_key
        UNIQUE (system_code),
    CONSTRAINT account_user_id_fkey
        FOREIGN KEY (user_id) REFERENCES identity.app_user(user_id) ON DELETE RESTRICT,
    CONSTRAINT account_leverage_check
        CHECK (leverage >= 1 AND leverage <= 10),
    CONSTRAINT cash_account_no_leverage
        CHECK (kind <> 'cash' OR leverage = 1.00),
    CONSTRAINT closed_status_consistent
        CHECK ((status = 'closed') = (closed_at IS NOT NULL)),
    CONSTRAINT short_requires_margin
        CHECK (NOT allow_short_selling OR kind = 'margin' OR is_system),
    CONSTRAINT system_account_shape
        CHECK (is_system AND user_id IS NULL AND system_code IS NOT NULL
         OR NOT is_system AND user_id IS NOT NULL AND system_code IS NULL)
);
```

trading.trade_order has the most constraints of any table, sixteen in all. fill_within_order stops a fill exceeding the order; order_shape_matches_kind checks that a limit order has a limit price and a stop order a stop price; rejected_has_reason makes the rejection reason mandatory exactly when the status is rejected. leaves_qty is a generated column, so the remaining size is always quantity minus filled_qty and can never drift out of step with them.

Listing 6.7 trading.trade_order

```
CREATE TABLE trading.trade_order (
    order_id            BIGINT GENERATED ALWAYS AS IDENTITY,
    client_order_id     UUID NOT NULL DEFAULT gen_random_uuid(),
    seq_no              BIGINT NOT NULL DEFAULT nextval('engine.order_seq'::regclass),
    account_id          BIGINT NOT NULL,
    instrument_id       INTEGER NOT NULL,
    side                trading.order_side NOT NULL,
    kind                trading.order_kind NOT NULL,
    tif                 trading.time_in_force NOT NULL DEFAULT 'day',
    quantity            qty_amt NOT NULL,
    limit_price         NUMERIC(20,8),
    stop_price          NUMERIC(20,8),
    trail_pct           pct,
    peak_price          NUMERIC(20,8),
    display_qty         NUMERIC(20,8),
    filled_qty          qty_amt0 NOT NULL DEFAULT 0,
```

```
    leaves_qty          NUMERIC(20,8) GENERATED ALWAYS AS (((quantity)::numeric -
        (filled_qty)::numeric)) STORED,
    avg_fill_price      NUMERIC(20,8),
    status              trading.order_status NOT NULL DEFAULT 'pending_new',
    stop_armed          BOOLEAN NOT NULL DEFAULT false,
    reject_reason       TEXT,
    parent_order_id     BIGINT,
    source_strategy_id  BIGINT,
    placed_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at          TIMESTAMPTZ,
    CONSTRAINT trade_order_pkey
        PRIMARY KEY (order_id),
    CONSTRAINT trade_order_client_order_id_key
        UNIQUE (client_order_id),
    CONSTRAINT fk_order_strategy
        FOREIGN KEY (source_strategy_id) REFERENCES social.strategy(strategy_id) ON DELETE
          SET NULL,
    CONSTRAINT trade_order_account_id_fkey
        FOREIGN KEY (account_id) REFERENCES trading.account(account_id),
    CONSTRAINT trade_order_instrument_id_fkey
        FOREIGN KEY (instrument_id) REFERENCES reference.instrument(instrument_id),
    CONSTRAINT trade_order_parent_order_id_fkey
        FOREIGN KEY (parent_order_id) REFERENCES trading.trade_order(order_id),
    CONSTRAINT avg_price_only_when_filled
        CHECK ((avg_fill_price IS NOT NULL) = (filled_qty > 0)),
    CONSTRAINT fill_within_order
        CHECK (filled_qty <= quantity),
    CONSTRAINT iceberg_display_fits
        CHECK (display_qty IS NULL OR display_qty <= quantity),
    CONSTRAINT market_order_tif
        CHECK (kind <> 'market' OR (tif = ANY (ARRAY['day', 'ioc', 'fok']))),
    CONSTRAINT no_self_parent
        CHECK (parent_order_id IS DISTINCT FROM order_id),
    CONSTRAINT order_shape_matches_kind
        CHECK (
CASE kind
    WHEN 'market' THEN limit_price IS NULL AND stop_price IS NULL AND trail_pct IS NULL
    WHEN 'limit' THEN limit_price IS NOT NULL AND stop_price IS NULL AND trail_pct IS NULL
    WHEN 'stop' THEN stop_price IS NOT NULL AND limit_price IS NULL AND trail_pct IS NULL
    WHEN 'stop_limit' THEN stop_price IS NOT NULL AND limit_price IS NOT NULL AND
        trail_pct IS NULL
    WHEN 'trailing_stop' THEN trail_pct IS NOT NULL AND limit_price IS NULL
    WHEN 'iceberg' THEN limit_price IS NOT NULL AND display_qty IS NOT NULL
    ELSE NULL
END),
    CONSTRAINT rejected_has_reason
        CHECK ((status = 'rejected') = (reject_reason IS NOT NULL)),
    CONSTRAINT trade_order_display_qty_check
        CHECK (display_qty IS NULL OR display_qty > 0),
    CONSTRAINT trade_order_limit_price_check
        CHECK (limit_price IS NULL OR limit_price > 0),
    CONSTRAINT trade_order_stop_price_check
        CHECK (stop_price IS NULL OR stop_price > 0)
);
```

trading.execution is partitioned by month on executed_at. PostgreSQL requires the partition key in every unique constraint of a partitioned table, so the primary key is (executed_at, execution_id). distinct_sides stops an order matching itself.

Listing 6.8 trading.execution

```
CREATE TABLE trading.execution (
    execution_id         BIGINT GENERATED ALWAYS AS IDENTITY,
    executed_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    instrument_id        INTEGER NOT NULL,
    price                price_amt NOT NULL,
    quantity             qty_amt NOT NULL,
    buy_order_id         BIGINT NOT NULL,
    sell_order_id        BIGINT NOT NULL,
    buy_account_id       BIGINT NOT NULL,
    sell_account_id      BIGINT NOT NULL,
    aggressor_side       trading.order_side NOT NULL,
    buy_fee              money_amt NOT NULL DEFAULT 0,
    sell_fee             money_amt NOT NULL DEFAULT 0,
    settlement_entry_id  BIGINT,
    CONSTRAINT execution_pkey
        PRIMARY KEY (executed_at, execution_id),
    CONSTRAINT execution_buy_account_id_fkey
        FOREIGN KEY (buy_account_id) REFERENCES trading.account(account_id),
    CONSTRAINT execution_buy_order_id_fkey
        FOREIGN KEY (buy_order_id) REFERENCES trading.trade_order(order_id),
```

```
    CONSTRAINT execution_instrument_id_fkey
        FOREIGN KEY (instrument_id) REFERENCES reference.instrument(instrument_id),
    CONSTRAINT execution_sell_account_id_fkey
        FOREIGN KEY (sell_account_id) REFERENCES trading.account(account_id),
    CONSTRAINT execution_sell_order_id_fkey
        FOREIGN KEY (sell_order_id) REFERENCES trading.trade_order(order_id),
    CONSTRAINT execution_settlement_entry_id_fkey
        FOREIGN KEY (settlement_entry_id) REFERENCES ledger.journal_entry(entry_id),
    CONSTRAINT distinct_sides
        CHECK (buy_order_id <> sell_order_id)
) PARTITION BY RANGE (executed_at);
```

Listing 6.9 trading.position

```
CREATE TABLE trading.position (
    account_id     BIGINT NOT NULL,
    instrument_id  INTEGER NOT NULL,
    quantity       money_amt NOT NULL DEFAULT 0,
    avg_cost       NUMERIC(20,8) NOT NULL DEFAULT 0,
    realized_pnl   money_amt NOT NULL DEFAULT 0,
    opened_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT position_pkey
        PRIMARY KEY (account_id, instrument_id),
    CONSTRAINT position_account_id_fkey
        FOREIGN KEY (account_id) REFERENCES trading.account(account_id) ON DELETE CASCADE,
    CONSTRAINT position_instrument_id_fkey
        FOREIGN KEY (instrument_id) REFERENCES reference.instrument(instrument_id),
    CONSTRAINT position_avg_cost_check
        CHECK (avg_cost >= 0)
);
```

6.3.3 Market data

market.quote holds one row per instrument and refuses a crossed book outright: book_not_crossed requires the best bid to be below the best ask. market.candle_1m checks the arithmetic of every bar. ohlc_coherent requires the high to be the highest of the four prices and the low the lowest, so a feed defect is rejected at write time instead of appearing later as an impossible candle on a chart. candle_source_known records whether a bar was built from live ticks or loaded as history, which Section 6.7 explains.

Listing 6.10 market.quote

```
CREATE TABLE market.quote (
    instrument_id    INTEGER NOT NULL,
    bid              NUMERIC(20,8),
    bid_size         qty_amt0 NOT NULL DEFAULT 0,
    ask              NUMERIC(20,8),
    ask_size         qty_amt0 NOT NULL DEFAULT 0,
    last_price       NUMERIC(20,8),
    last_size        qty_amt0 NOT NULL DEFAULT 0,
    day_open         NUMERIC(20,8),
    day_high         NUMERIC(20,8),
    day_low          NUMERIC(20,8),
    prev_close       NUMERIC(20,8),
    day_volume       qty_amt0 NOT NULL DEFAULT 0,
    reference_price  NUMERIC(20,8),
    updated_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT quote_pkey
        PRIMARY KEY (instrument_id),
    CONSTRAINT quote_instrument_id_fkey
        FOREIGN KEY (instrument_id) REFERENCES reference.instrument(instrument_id) ON
          DELETE CASCADE,
    CONSTRAINT book_not_crossed
        CHECK (bid IS NULL OR ask IS NULL OR bid < ask),
    CONSTRAINT quote_ask_check
        CHECK (ask IS NULL OR ask > 0),
    CONSTRAINT quote_bid_check
        CHECK (bid IS NULL OR bid > 0),
    CONSTRAINT quote_last_price_check
```

```
        CHECK (last_price IS NULL OR last_price > 0)
);
```

Listing 6.11 market.candle_1m

```
CREATE TABLE market.candle_1m (
    instrument_id  INTEGER NOT NULL,
    bucket_start   TIMESTAMPTZ NOT NULL,
    open           price_amt NOT NULL,
    high           price_amt NOT NULL,
    low            price_amt NOT NULL,
    close          price_amt NOT NULL,
    volume         qty_amt0 NOT NULL DEFAULT 0,
    trade_count    INTEGER NOT NULL DEFAULT 0,
    vwap           NUMERIC(20,8),
    source         TEXT NOT NULL DEFAULT 'tick',
    CONSTRAINT candle_1m_pkey
        PRIMARY KEY (instrument_id, bucket_start),
    CONSTRAINT candle_1m_instrument_id_fkey
        FOREIGN KEY (instrument_id) REFERENCES reference.instrument(instrument_id),
    CONSTRAINT bucket_aligned
        CHECK (date_trunc('minute', bucket_start) = bucket_start),
    CONSTRAINT candle_1m_trade_count_check
        CHECK (trade_count >= 0),
    CONSTRAINT candle_source_known
        CHECK (source = ANY (ARRAY['tick', 'backfill', 'provider'])),
    CONSTRAINT ohlc_coherent
        CHECK (high >= low AND high >= open AND high >= close AND low <= open AND low <=
          close)
) PARTITION BY RANGE (bucket_start);
```

6.3.4 The ledger

A journal entry holds no amounts; its postings do. In ledger.posting, asset_instrument_ agreement makes a malformed leg impossible to write: a cash leg must have a currency and no instrument, a security leg an instrument and no currency. ledger.balance has no primary key, for the reason given in Section 5.5.1; its key is the NULLS NOT DISTINCT unique index shown with it.

Listing 6.12 ledger.journal_entry

```
CREATE TABLE ledger.journal_entry (
    entry_id           BIGINT GENERATED ALWAYS AS IDENTITY,
    kind               ledger.entry_kind NOT NULL,
    occurred_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    description        TEXT NOT NULL,
    source_kind        TEXT,
    source_id          BIGINT,
    reverses_entry_id  BIGINT,
    created_by         BIGINT,
    CONSTRAINT journal_entry_pkey
        PRIMARY KEY (entry_id),
    CONSTRAINT journal_entry_created_by_fkey
        FOREIGN KEY (created_by) REFERENCES identity.app_user(user_id),
    CONSTRAINT journal_entry_reverses_entry_id_fkey
        FOREIGN KEY (reverses_entry_id) REFERENCES ledger.journal_entry(entry_id),
    CONSTRAINT no_self_reversal
        CHECK (reverses_entry_id IS DISTINCT FROM entry_id)
);
```

Listing 6.13 ledger.posting

```
CREATE TABLE ledger.posting (
    posting_id     BIGINT GENERATED ALWAYS AS IDENTITY,
    entry_id       BIGINT NOT NULL,
    account_id     BIGINT NOT NULL,
    asset_class    ledger.asset_class NOT NULL,
    instrument_id  INTEGER,
```

```
    amount         money_amt NOT NULL,
    memo           TEXT,
    currency       CHAR(3),
    CONSTRAINT posting_pkey
        PRIMARY KEY (posting_id),
    CONSTRAINT posting_account_id_fkey
        FOREIGN KEY (account_id) REFERENCES trading.account(account_id),
    CONSTRAINT posting_currency_fk
        FOREIGN KEY (currency) REFERENCES reference.currency(code),
    CONSTRAINT posting_entry_id_fkey
        FOREIGN KEY (entry_id) REFERENCES ledger.journal_entry(entry_id) ON DELETE
          CASCADE,
    CONSTRAINT posting_instrument_id_fkey
        FOREIGN KEY (instrument_id) REFERENCES reference.instrument(instrument_id),
    CONSTRAINT asset_instrument_agreement
        CHECK (asset_class = 'cash' AND instrument_id IS NULL AND currency IS NOT NULL
         OR asset_class = 'security' AND instrument_id IS NOT NULL AND currency IS NULL),
    CONSTRAINT posting_amount_check
        CHECK (amount <> 0)
);
```

Listing 6.14 ledger.balance

```
CREATE TABLE ledger.balance (
    account_id     BIGINT NOT NULL,
    asset_class    ledger.asset_class NOT NULL,
    instrument_id  INTEGER,
    balance        money_amt NOT NULL DEFAULT 0,
    updated_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    currency       CHAR(3),
    CONSTRAINT balance_account_id_fkey
        FOREIGN KEY (account_id) REFERENCES trading.account(account_id) ON DELETE CASCADE,
    CONSTRAINT balance_instrument_id_fkey
        FOREIGN KEY (instrument_id) REFERENCES reference.instrument(instrument_id)
);

CREATE UNIQUE INDEX uq_balance_slot
    ON ledger.balance (account_id, asset_class, currency, instrument_id) NULLS NOT
        DISTINCT;
```

6.4 The constraint inventory

Counted across the whole schema rather than the twelve tables above, the database declares the following. The figures are read from pg_constraint and information_schema, so they describe the database as built.

Listing 6.15 Declared constraints by kind

```
-- Declared constraints, counted by kind
        kind         | declared
---------------------+----------
 NOT NULL            |      540
 CHECK               |       71
 FOREIGN KEY         |       57
 PRIMARY KEY         |       37
 UNIQUE              |       11
 EXCLUDE             |        1
 CONSTRAINT TRIGGER  |        1
(7 rows)
```

6.5 INSERT statements and sample data

The demonstration database is filled by a seeding programme, scripts/seed.py, because the history has to hang together: every candle must agree with the ticks behind it, every execution needs a balanced journal entry, and every position must equal the sum of its postings. The

programme issues ordinary SQL, and Listing 6.16 is a representative sequence. It adds an Indian exchange with two listed instruments, a user, a rupee account, and the deposit that funds it.

Listing 6.16 INSERT statements

```
BEGIN;
INSERT INTO reference.exchange (code, name, country_code, timezone, currency,
                                session_open, session_close)
VALUES ('XBOM', 'BSE Limited', 'IN', 'Asia/Kolkata', 'INR', '09:15', '15:30')
RETURNING exchange_id, code, name, timezone;

INSERT INTO reference.instrument (symbol, exchange_id, name, sector, currency,
                                  tick_size, lot_size, price_band_pct)
SELECT v.sym, e.exchange_id, v.nm, v.sec, 'INR', 0.05, 1, 20.0
  FROM reference.exchange e,
       (VALUES ('RELIANCE', 'Reliance Industries Ltd',       'Energy'),
               ('TCS',      'Tata Consultancy Services Ltd', 'Technology')) v(sym, nm,
          sec)
 WHERE e.code = 'XBOM'
RETURNING instrument_id, symbol, currency, tick_size;

INSERT INTO identity.app_user (email, password_hash, full_name, kyc_state,
    kyc_verified_at)
VALUES ('demo.student@apex.io', identity.hash_password('Apex@2026'),
        'Demo Student', 'verified', now())
RETURNING user_id, email, kyc_state, left(password_hash, 7) AS hash_prefix;

INSERT INTO trading.account (account_no, user_id, kind, base_currency)
SELECT 'APX-DEMO01', user_id, 'cash', 'INR'
  FROM identity.app_user WHERE email = 'demo.student@apex.io'
RETURNING account_id, account_no, kind, base_currency;

SELECT ledger.post_entry('deposit', 'Opening balance', ARRAY[
         ROW(1, 'cash', NULL, -100000.00, 'funding')::ledger.leg,
         ROW((SELECT account_id FROM trading.account WHERE account_no = 'APX-DEMO01'),
             'cash', NULL, 100000.00, 'funding')::ledger.leg],
       p_currency => 'INR') AS entry_id;

SELECT a.account_no, b.asset_class, b.currency, b.balance
  FROM ledger.balance b JOIN trading.account a USING (account_id)
 WHERE a.account_no = 'APX-DEMO01';
ROLLBACK;
```

Listing 6.17 is what the server returned when that script was run. Each statement passed every constraint of Section 6.3. Two details are worth noting. The stored password begins $2a$10$, the marker of a bcrypt hash with cost factor 10, so the plain text never reaches the table. And the balance in part (f) was never inserted by anyone: the trigger on ledger.posting derived it from the two legs of the deposit.

Listing 6.17 Output of the INSERT statements

```
-- Sample data, inserted for real and then rolled back
-- (a) A new exchange: BSE, Mumbai
 exchange_id | code |    name     |   timezone
-------------+------+-------------+--------------
         145 | XBOM | BSE Limited | Asia/Kolkata
(1 row)

-- (b) Two instruments listed on it, priced in rupees
 instrument_id |  symbol  | currency | tick_size
---------------+----------+----------+------------
           613 | RELIANCE | INR      | 0.05000000
           614 | TCS      | INR      | 0.05000000
(2 rows)

-- (c) A user. The password is hashed by pgcrypto; only the hash is stored.
 user_id |        email         | kyc_state | hash_prefix
---------+----------------------+-----------+-------------
     350 | demo.student@apex.io | verified  | $2a$10$
(1 row)
```

```
-- (d) A rupee cash account for that user
 account_id | account_no | kind | base_currency
------------+------------+------+---------------
        383 | APX-DEMO01 | cash | INR
(1 row)

-- (e) Funding is a journal entry, not an UPDATE: money moves from the bank
 entry_id
----------
     5889
(1 row)

-- (f) The balance that results, derived by trigger from the postings
 account_no | asset_class | currency |     balance
------------+-------------+----------+-----------------
 APX-DEMO01 | cash        | INR      | 100000.00000000
(1 row)

-- ROLLBACK: the demonstration leaves nothing behind.
```

6.6 The resulting database

The seeded demonstration database covers forty-five trading sessions. Listings 6.18 and 6.19 describe what it contains; both were produced by querying the running instance, not by transcription.

Listing 6.18 Objects and on-disk size, by schema

```
-- The schemas and what they hold, read back from the catalogue
  schema   | tables | partitioned | views | on_disk
-----------+--------+-------------+-------+---------
 analytics |      1 |           0 |     8 | 264 kB
 audit     |      1 |           0 |     0 | 936 kB
 identity  |      3 |           0 |     0 | 288 kB
 ledger    |      3 |           0 |     0 | 512 kB
 market    |      1 |           2 |     1 | 61 MB
 ops       |      3 |           0 |     3 | 88 kB
 reference |      6 |           0 |     0 | 760 kB
 risk      |      5 |           0 |     0 | 248 kB
 social    |      7 |           0 |     1 | 376 kB
 surveil   |      2 |           0 |     0 | 136 kB
 trading   |      3 |           1 |     1 | 1824 kB
(11 rows)
```

Listing 6.19 Row counts of the seeded demonstration data

```
-- Row counts of the seeded demonstration data
       relation       | count
----------------------+--------
 audit.activity_log   |    708
 identity.app_user    |      8
 ledger.balance       |    149
 ledger.journal_entry |    258
 ledger.posting       |   1135
 market.candle_1m     | 442468
 market.tick          |   4592
 reference.currency   |     10
 reference.fx_rate    |    414
 reference.instrument |     24
 trading.account      |     13
 trading.execution    |    175
 trading.position     |    124
 trading.trade_order  |    600
(14 rows)
```

Four hundred and forty-two thousand minute bars across twenty-four instruments is the figure that justifies the partitioning and indexing work described in Section 8.5: at that volume the

chart query is the hot path of the whole application, and the difference between a sequential scan and a pruned index scan is the difference between a usable chart and an unusable one.

6.7 Live market data

Seeded history is only half the data. An ingestion worker polls Yahoo Finance every few seconds, normalises each response into a tick, and the minute bars are aggregated from those ticks. Figure 6.1 traces that path. It also records the two defects that reached the running system through it, because both were instructive and both produced wrong data rather than an error message.

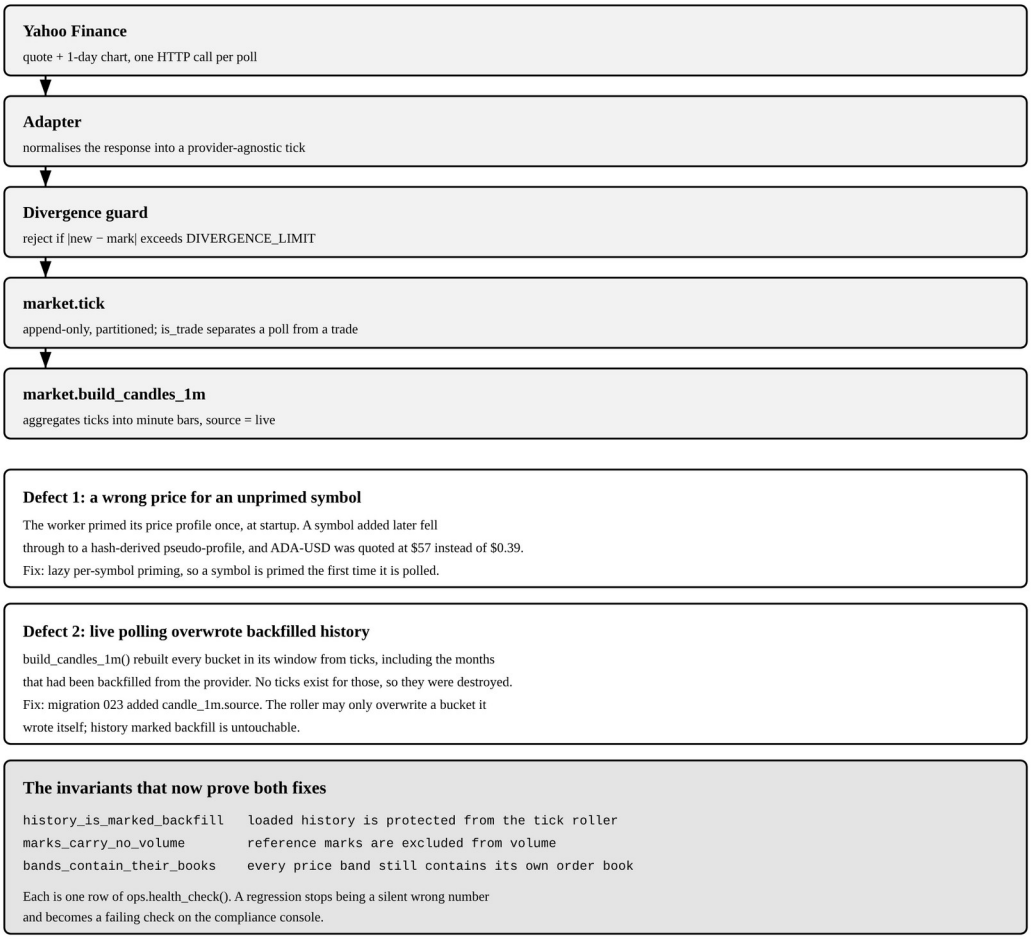


Figure 6.1 The market-data path, and the two guards added after it broke

The second defect is the sharper lesson. market.build_candles_1m rebuilt every bucket in its window from the tick table, which was correct for live minutes and destructive for the months that had been loaded from the provider, since no ticks exist for those. Migration 023 added candle_1m.source and the rule that the roller may only overwrite a bucket it wrote itself. The repair is not the interesting part; the invariant that now fails loudly if it ever regresses is.

7. DATABASE OPERATIONS AND OUTPUTS

Every listing in this section is a verbatim transcript: the statement was run against the seeded demonstration database through psql and the output beneath it is what the server printed. Statements that modify data are wrapped in an explicit transaction and rolled back, so the demonstration leaves the database exactly as it found it. Two presentational changes have been applied: the absolute path psql prints in an error prefix is shortened to the file name, and a line wider than the page is folded with a hanging indent.

7.1 Projection, selection, ordering and row limiting

The security master is the simplest read in the system and already exercises projection, a compound predicate, ordering on two keys and a row limit.

Listing 7.1 SELECT with WHERE, IN, ORDER BY and LIMIT

```
-- Projection, filtering, ordering and row limiting
 symbol |          name           |  kind  | currency | tick_size  | price_band_pct
--------+-------------------------+--------+----------+------------+----------------
 AAPL   | Apple Inc.              | equity | USD      | 0.01000000 |        20.0000
 AMD    | Advanced Micro Devices  | equity | USD      | 0.01000000 |        25.0000
 AMZN   | Amazon.com Inc.         | equity | USD      | 0.01000000 |        20.0000
 DIS    | The Walt Disney Company | equity | USD      | 0.01000000 |        20.0000
 GOOGL  | Alphabet Inc.           | equity | USD      | 0.01000000 |        20.0000
 INTC   | Intel Corporation       | equity | USD      | 0.01000000 |        20.0000
 JPM    | JPMorgan Chase & Co.    | equity | USD      | 0.01000000 |        15.0000
 META   | Meta Platforms Inc.     | equity | USD      | 0.01000000 |        20.0000
(8 rows)
```

7.2 Joins across bounded contexts

A trader’s portfolio cannot be answered from one table. The query below joins five tables drawn from four different schemas (position and account from trading, app_user from identity, instrument from reference and quote from market) and computes unrealised profit from the difference between the live mark and the average cost. The join is the reason the data is normalised: the average cost lives in exactly one place, and the live price in exactly one other.

Listing 7.2 A five-table inner join with a derived column

```
-- Five-table inner join across four bounded contexts
  trader   | account_no | symbol  |  qty   | avg_cost | last_px | unrealised
-----------+------------+---------+--------+----------+---------+------------
 Sam       | APX-000345 | MSFT    |  48.00 |   505.24 |  751.22 |   11807.04
 Samrender | APX-000342 | TSLA    |  58.00 |   383.71 |  561.33 |   10302.07
 Vartika   | APX-000343 | QQQ     |  53.00 |   493.49 |  673.05 |    9516.93
 Samrender | APX-000342 | SOL-USD |  62.26 |   192.75 |  313.62 |    7524.98
 Ginni     | APX-000346 | TSLA    |  21.00 |   235.88 |  561.33 |    6834.45
 Kumar     | APX-000344 | QQQ     |  30.00 |   464.38 |  673.05 |    6260.04
 Vartika   | APX-000343 | SPY     |  87.00 |   555.51 |  617.20 |    5366.84
 Samrender | APX-000342 | V       | 171.00 |   281.71 |  298.58 |    2883.98
(8 rows)
```

7.3 Aggregate functions

GROUP BY with HAVING answers the question a trading desk actually asks: which instruments carry enough order flow to be worth attention. The aggregate FILTER clause is used in preference to a CASE expression inside count() because it states the intention directly and the planner can evaluate all four aggregates in one pass over the grouped rows.

Listing 7.3 count, sum, avg, GROUP BY, HAVING and the aggregate FILTER clause

```
-- Aggregates with GROUP BY, HAVING and FILTER
 symbol | orders | buys | sells | filled | avg_px
--------+--------+------+-------+--------+--------
 DIS    |     54 |   33 |    21 |   2400 |  98.74
 INTC   |     48 |   24 |    24 |   9748 |  43.05
 NVDA   |     36 |   18 |    18 |   2518 |  84.89
 AMZN   |     35 |   18 |    17 |   1112 | 177.28
 AAPL   |     35 |   20 |    15 |    934 | 186.33
 AMD    |     32 |   16 |    16 |   1090 | 166.98
 SPY    |     32 |   16 |    16 |    358 | 565.13
 GOOGL  |     30 |   15 |    15 |   1306 | 157.58
(8 rows)
```

7.4 Subqueries

Three kinds of subquery appear in one statement. The order count is a scalar subquery, evaluated once per outer row; the latest order timestamp is a correlated subquery, referring to the outer user; and the WHERE clause uses EXISTS, which stops at the first matching row rather than counting them all.

Listing 7.4 Scalar, correlated and EXISTS subqueries

```
-- Scalar, correlated and EXISTS subqueries in one statement
       full_name       | orders |          last_order
-----------------------+--------+-------------------------------
 Samrender Singh Saini |     55 | 2026-10-18 23:09:12.364784+00
 Vartika Chaudhary     |     47 | 2026-10-18 22:58:08.626855+00
 Ginni Kaur            |     32 | 2026-10-18 19:57:48.123864+00
 Kumar Paritosh        |     30 | 2026-10-16 18:11:03+00
 Pari Sharma           |     19 | 2026-10-18 22:57:48.198088+00
 Sam Saini             |     16 | 2026-10-18 22:58:08.396372+00
(6 rows)
```

7.5 Common table expressions and window functions

A running cash balance is a classic case for a window function: the answer for each row depends on every earlier row in the same partition. Computing it with a correlated subquery would be quadratic; the window frame ROWS UNBOUNDED PRECEDING computes it in a single ordered pass. The common table expression exists only to name the intermediate result, which keeps the window clauses readable.

Listing 7.5 A CTE feeding sum() OVER and rank() OVER

```
-- CTE + window functions: running cash position for one account
  on_date   | currency |  amount   | running_cash | size_rank
------------+----------+-----------+--------------+-----------
 2026-08-16 | USD      | 500000.00 |    500000.00 |         1
 2026-08-17 | USD      | -19964.69 |    480035.31 |        11
 2026-08-20 | USD      | -19997.70 |    460037.61 |        10
 2026-08-21 | USD      |  -7796.54 |    452241.07 |        28
 2026-08-25 | USD      | -20026.09 |    432214.98 |         6
 2026-08-25 | USD      |  -2049.28 |    430165.71 |        42
 2026-08-28 | USD      |  -2049.34 |    428116.37 |        41
 2026-08-31 | USD      | -12007.16 |    416109.21 |        16
(8 rows)
```

7.6 INSERT, UPDATE and DELETE

The three data-modifying statements are shown together inside one transaction, each with a RETURNING clause so the effect is visible without a second query, and the transaction is then rolled back. RETURNING also hands back the generated identity key in the same statement, so no second query is needed to find out which row was written.

Listing 7.6 INSERT, UPDATE and DELETE with RETURNING, undone by ROLLBACK

```
-- INSERT / UPDATE / DELETE inside one transaction, then ROLLBACK
 watchlist_id | instrument_id |         note
--------------+---------------+----------------------
          356 |           601 | added for the report
(1 row)

 account_id |  new_cap
------------+------------
        375 | 1100000.00
(1 row)

 instrument_id |         note
---------------+----------------------
           601 | added for the report
(1 row)

-- ROLLBACK issued: none of the three statements above is durable.
```

7.7 The complete order round trip

This is the operation the whole database exists to perform. Two orders are submitted through engine.submit_order; the engine validates them, matches them against the resting book, writes an execution row, posts the settling journal entry and updates the position, all inside the one transaction that the function opened. Nothing in the listing below was done by application code.

The first order is instructive. It asks to sell at 150.00, yet it fills at 174.73. That is not an error: price-time priority awards the aggressor the best resting price on the opposite side of the book, which here was a bid well above the seller’s own limit. The second order bids below the best ask, does not cross, and therefore rests on the book in status working.

Listing 7.7 Submission, matching, settlement and cancellation, end to end

```
-- A complete order round trip executed entirely inside the database
-- (a) The Vartika account sends a limit SELL of 10 AAPL, asking 150.00.
--     The book already holds a better bid, so price-time priority
--     fills the aggressor AT THE RESTING PRICE, not at its own limit.
 order_id | status | filled_qty  | avg_fill_price | reject_reason
----------+--------+-------------+----------------+---------------
     9911 | filled | 10.00000000 |   174.73000000 |
(1 row)

-- (b) The Samrender account bids 155.00, below the best ask, so the
--     order rests on the book in status working instead of filling.
 order_id | status
----------+---------
 9912     | working
(1 row)

-- (c) The execution row the engine wrote for (a)
 execution_id | price  |  qty  | aggressor_side | buyer | seller
--------------+--------+-------+----------------+-------+--------
         4111 | 174.73 | 10.00 | sell           |     4 |    376
(1 row)

-- (d) Every ledger leg that settled it, and their sum
 account_id | asset_class | sym  | currency |  amount
------------+-------------+------+----------+----------
          4 | cash        | -    | USD      | -1747.65
        376 | cash        | -    | USD      |  1745.55
          4 | security    | AAPL |          |    10.00
        376 | security    | AAPL |          |   -10.00
          2 | cash        | -    | USD      |     2.10
(5 rows)

 asset_class | currency | must_be_zero
-------------+----------+--------------
 cash        | USD      |   0.00000000
 security    |          |   0.00000000
(2 rows)

-- (e) The resting order from (b) is cancelled by the engine
 cancel_order
--------------
 cancelled
(1 row)
```

The five postings in part (d) are the double entry. The buying account gives up cash and receives stock; the selling account gives up stock and receives cash net of commission; the fee account receives the commission. The final query proves the point the constraint will enforce at COMMIT: summed per asset class, the legs are exactly zero. Figure 7.1 shows the same entry as a diagram.

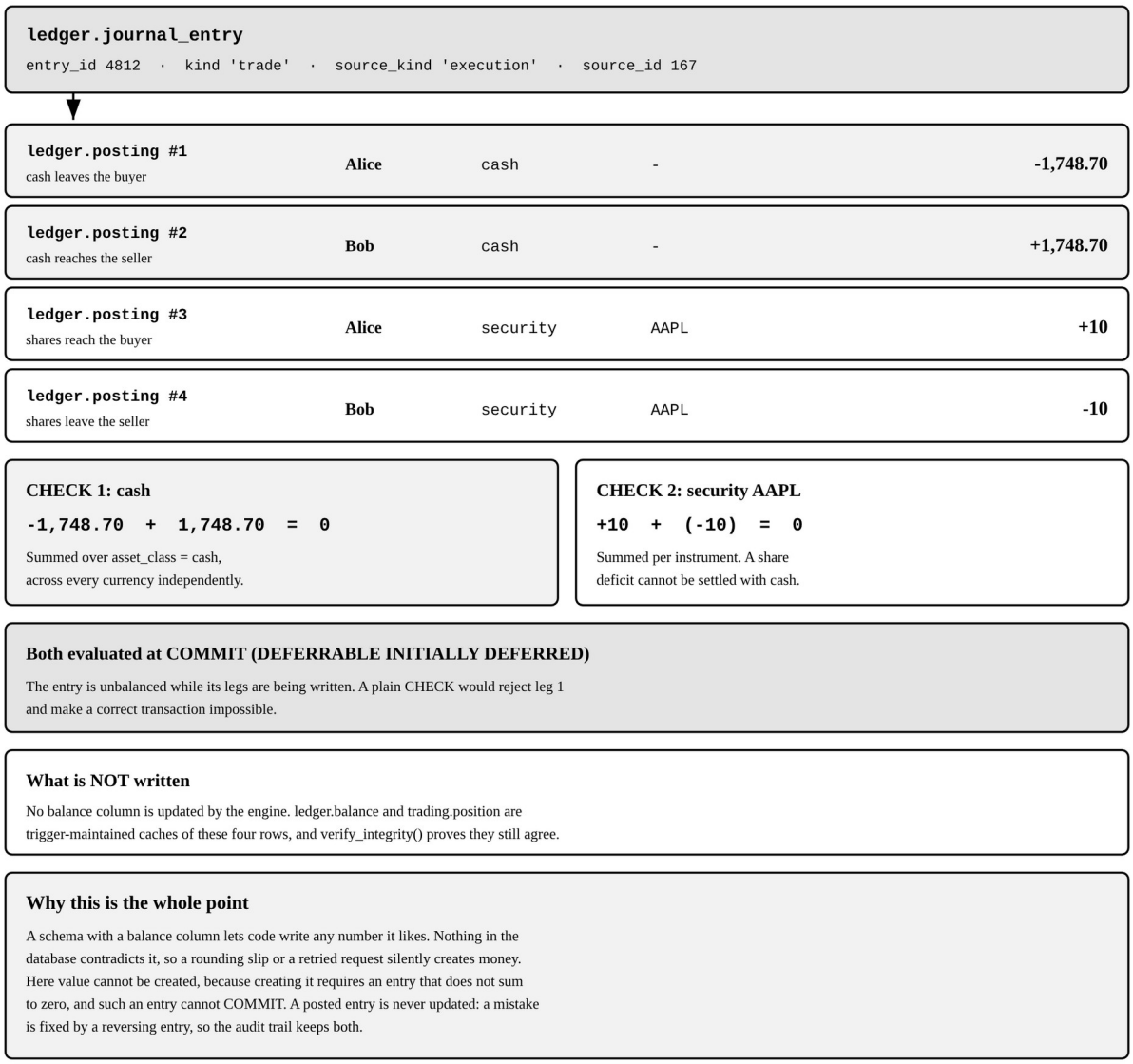


Figure 7.1 One purchase, four principal legs, two zero-sum checks

7.8 Operations the database refuses

The three orders below are each rejected before any row is written to the book, by risk.pretrade_check, and the rejection reason is returned to the caller rather than raised as an exception, because a rejected order is a normal business outcome and not a fault.

Listing 7.8 Short-sale gate, notional limit and price band, each enforced in SQL

```
-- Pre-trade risk: three orders the database refuses to accept.
--      Trailing zeros are stripped from the messages for width.
-- (a) Selling stock the account does not hold, short selling off
  status  |                                reject_reason
----------+-------------------------------------------------------------------------------
 rejected | cannot sell 5 NVDA: you hold 0. Short selling is not enabled on this account.
(1 row)

-- (b) A buy far beyond the single-order notional cap
  status  |                       reject_reason
----------+-------------------------------------------------------------
 rejected | order value 20000000 USD exceeds your limit of 1000000 USD
(1 row)

-- (c) A limit price outside the instrument price band
  status  |                reject_reason
----------+-------------------------------------------------
 rejected | price 20 outside 20% band [136.8080, 205.2120]
(1 row)
```

7.9 A constraint that fires at COMMIT

The zero-sum rule cannot be checked after each statement, because the legs of an entry are inserted one row at a time and the entry is legitimately unbalanced while that is happening. It is therefore declared DEFERRABLE INITIALLY DEFERRED. The consequence is visible below: the INSERT succeeds, and the COMMIT fails.

Listing 7.9 A deferred constraint trigger rejecting an unbalanced entry at COMMIT

```
-- A deliberately unbalanced journal entry: the deferred
--      constraint lets the INSERT succeed and fails the COMMIT.
-- The INSERT below is accepted: the check is DEFERRED.
 posting_id | account_id |    amount
------------+------------+---------------
      25187 |        375 | 1000.00000000
(1 row)

-- COMMIT now runs the constraint trigger:
psql:q09_defer.sql:13: ERROR:  ledger entry 5888 has 1 posting(s): double-entry requires
    at least two
HINT:  value must move between two named accounts
CONTEXT:  PL/pgSQL function ledger.assert_entry_balanced() line 11 at RAISE
```

This is how the ledger is guaranteed to balance. The guarantee does not depend on the application being careful or on a nightly reconciliation job; the database will not let any transaction commit an unbalanced entry.

7.10 Query plans

The chart query reads one instrument over one day from a table holding more than four hundred thousand minute bars spread over eleven monthly partitions. EXPLAIN ANALYZE confirms that the planner opens exactly one of them and reaches the hundred rows it needs through a backward index scan, so no sort is required for the descending order and no row outside October is ever examined.

Listing 7.10 Partition pruning and a backward index scan, measured

```
-- EXPLAIN ANALYZE on the chart query: 11 monthly partitions
--       exist; the planner opens exactly one.
                                                  QUERY PLAN
-------------------------------------------------------------------------------------------------
 Limit (actual rows=100 loops=1)
   ->  Index Scan Backward using candle_1m_p202610_pkey on candle_1m_p202610 candle_1m
       (actual rows=100 loops=1)
         Index Cond: ((instrument_id = 589) AND (bucket_start >= '2026-10-01
          00:00:00+00'::timestamp with time zone) AND (bucket_start < '2026-10-02
          00:00:00+00'::timestamp with time zone))
 Planning Time: 1.377 ms
 Execution Time: 0.976 ms
(5 rows)
```

Figure 8.2, in the section on physical design, places this result in context together with the index strategy and the measured latency of the other hot paths.

7.11 The database checking itself

ops.health_check() is a single function that evaluates eighteen invariants and returns one row per invariant. It runs against the live data, not a test fixture, data and is exposed through the application's health endpoint. Several of the defects found during development produced no error message and no exception; they were found because one of these checks began to report a non-zero count.

Listing 7.11 ops.health_check(): eighteen invariants, evaluated against live data

```
-- The eighteen invariants the database checks on itself
          check_name             |  status   |                  detail
---------------------------------+-----------+-----------------------------------------
 open_critical_alerts            | ATTENTION | 1 open critical alert(s)
 balance_cache_matches_journal   | PASS      | 0 drifted slot(s)
 bands_contain_their_books       | PASS      | every price band still contains its own
 closed_system_conservation      | PASS      | 0 non-conserved asset(s)
 copy_queue_backlog              | PASS      | 6 unprocessed fan-out job(s)
 every_book_has_a_quote          | PASS      | all 24 books are quoted
 every_cash_posting_has_currency | PASS      | 0 posting(s) without a currency
 every_entry_balances            | PASS      | 0 unbalanced entry/entries
 filled_orders_have_fills        | PASS      | 0 order(s) marked filled with no executi
 history_is_marked_backfill      | PASS      | loaded history is protected from the tic
 marks_carry_no_volume           | PASS      | reference marks are excluded from volume
 no_crossed_books                | PASS      | 0 crossed book(s)
 no_overfilled_orders            | PASS      | 0 order(s) filled beyond quantity
 no_unauthorised_overdraft       | PASS      | 0 overdrawn account(s)
 partition_runway                | PASS      | 4 future tick partition(s) available
 positions_match_ledger          | PASS      | 0 mismatched position(s)
 resting_prices_on_tick_grid     | PASS      | every resting price is a whole number of
 views_respect_rls               | PASS      | every view over user data runs as the ca
(18 rows)
```

The one row reading ATTENTION is the surveillance subsystem doing its work: the seeded order flow contains a deliberate wash-trade pattern, the scanner found it, and the resulting critical alert is open and awaiting a compliance decision. A system that reported no alerts here would mean the scanner was not running.

8. ADVANCED DATABASE FEATURES

The guidelines list procedures, functions, triggers, views and transaction control as the advanced features expected of a relational project. All five are present, and this section adds the three that the design actually depends on most heavily: Row Level Security, declarative partitioning with a deliberate index strategy, and an append-only audit trail. In each case the feature is shown working rather than merely declared.

8.1 Stored functions and procedures

Eighty-three functions and one procedure are defined across the twelve schemas. The business rules live in them rather than in the application. Table 8.1 lists the ones a reader needs in order to follow the system.

Table 8.1 The principal stored routines

Routine	Language	Responsibility
engine.submit_order	PL/pgSQL	Validate, risk-check, persist, lock the book and match an order
engine.match_order	PL/pgSQL	Price-time priority matching loop against the resting book
engine.settle_fill	PL/pgSQL	Write the execution row and post the settling journal entry
engine.elect_stops	PL/pgSQL	Arm and elect stop and trailing-stop orders after a price move
ledger.post_entry	PL/pgSQL	Write a journal entry and its legs as one atomic unit
ledger.verify_integrity	PL/pgSQL	Re-derive every balance from the journal and report drift
risk.pretrade_check	PL/pgSQL	Buying power, short-sale gate, notional cap, price band, halts
market.candles	SQL	Serve any chart interval from the one-minute base table
market.candles_with_indicators	SQL	SMA, EMA, Bollinger bands and RSI by window function
analytics.performance_stats	SQL	Return, drawdown, Sharpe ratio and win rate for an account
surveil.run_all_scans	PL/pgSQL	Wash trading, spoofing, momentum ignition, volume anomaly
ops.health_check	SQL	Evaluate the eighteen invariants of Section 7.11
ops.end_of_day	PROCEDURE	Expire orders, snapshot NAV, refresh matviews, roll partitions

ops.end_of_day is declared as a PROCEDURE rather than a function for a specific reason: a procedure may issue COMMIT, and the end-of-day run is a sequence of independent steps that should not be undone as a group if the last of them fails.

8.1.1 The matching loop

The core of engine.match_order is reproduced below. Two decisions in it matter most. The ORDER BY clause is price-time priority expressed as SQL, and it is served directly by a partial index so no sort step is required. The FOR UPDATE clause is deliberately not FOR UPDATE SKIP LOCKED: skipping a locked order would let a later order jump the queue, which is precisely the fairness property the exchange exists to guarantee.

Listing 8.1 Price-time priority, expressed as an ORDER BY an index can serve

```
-- Find the single best contra order.
-- The ORDER BY here *is* price-time priority, and it is served directly
-- by ix_book_bids / ix_book_asks with no sort step.
-- SKIP LOCKED is deliberately NOT used: skipping a locked order would
-- violate time priority by letting a later order jump the queue, so we
-- block and preserve fairness instead.
SELECT c.order_id, c.limit_price, c.leaves_qty, c.account_id
  INTO v_contra
  FROM trading.trade_order c
 WHERE c.instrument_id = v_o.instrument_id
   AND c.side   = 'sell'
   AND c.status IN ('working','partially_filled')
   AND NOT c.stop_armed
   AND c.limit_price IS NOT NULL
   AND c.leaves_qty  > 0
   AND (v_limit IS NULL OR c.limit_price <= v_limit)
 ORDER BY c.limit_price ASC, c.seq_no ASC
 LIMIT 1
 FOR UPDATE;
```

Time priority is taken from seq_no, a value drawn from a dedicated sequence, and not from the timestamp. Two orders placed in the same millisecond would tie on a timestamp, and a clock adjusted backwards by NTP would reorder them; a sequence does neither. Figure 8.1 traces the complete lifecycle of which this loop is stage five.

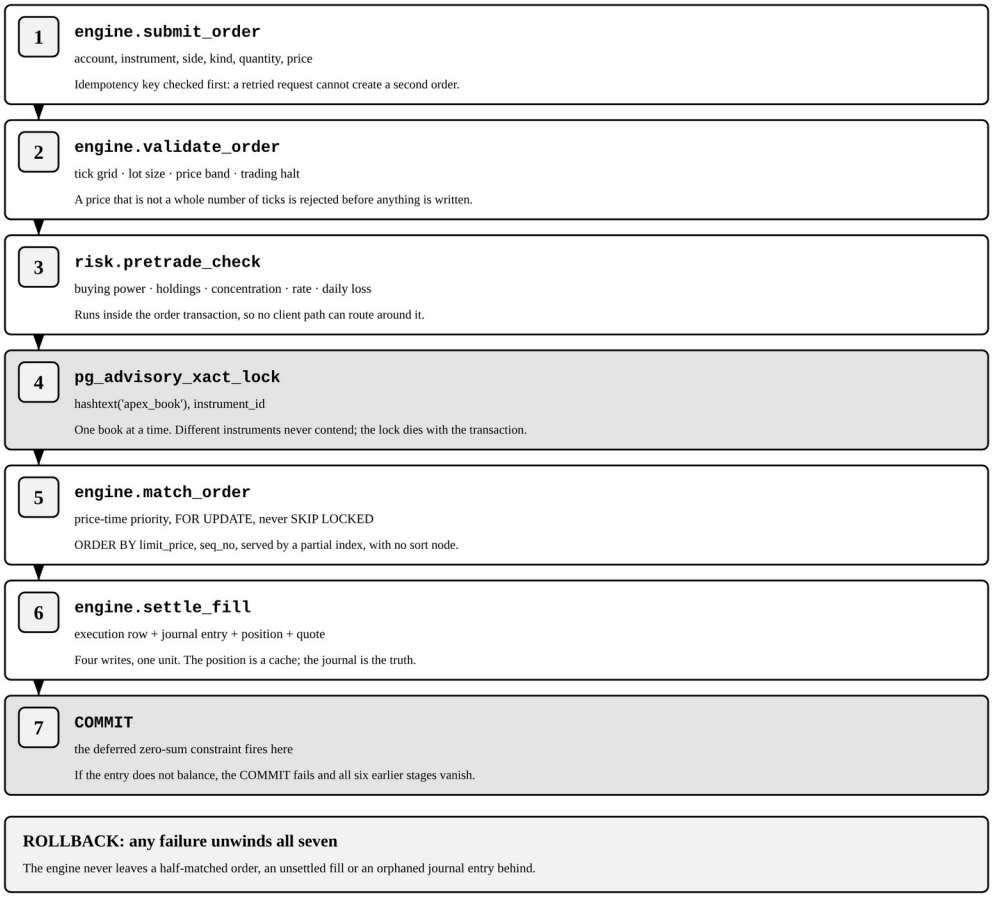


Figure 8.1 The seven stages an order passes through, all inside one transaction

8.2 Triggers

Fifteen user-defined triggers are declared on the base tables. They fall into four groups: those that maintain derived state, those that enforce an invariant, those that make a table append-only, and those that write the audit trail.

Listing 8.2 Trigger inventory, read from pg_trigger

```
-- Trigger inventory, read back from the catalogue
        table_name         |          trigger_name           |    kind
---------------------------+---------------------------------+------------
 identity.app_user         | trg_audit_user                  | row
 identity.app_user         | trg_user_default_watchlist      | row
 ledger.journal_entry      | trg_journal_immutable           | row
 ledger.posting            | trg_posting_balanced            | constraint
 ledger.posting            | trg_posting_immutable           | row
 ledger.posting            | trg_posting_syncs_position      | row
 ledger.posting            | trg_posting_updates_balance     | row
 social.copy_subscription  | trg_no_self_copy                | row
 surveil.alert             | trg_audit_alert                 | row
 trading.account           | trg_account_default_limits      | row
 trading.account           | trg_audit_account               | row
 trading.execution         | trg_block_self_match            | row
 trading.execution         | trg_execution_enqueues_copies   | row
 trading.trade_order       | trg_audit_order_material        | row
 trading.trade_order       | trg_audit_order_write           | row
(15 rows)
```

One of the fifteen is a CONSTRAINT trigger rather than an ordinary row trigger, and the distinction is the whole of Section 7.9. Its declaration is reproduced below.

Listing 8.3 The deferred constraint trigger that guarantees double entry

```
CREATE CONSTRAINT TRIGGER trg_posting_balanced
    AFTER INSERT OR UPDATE OR DELETE ON ledger.posting
    DEFERRABLE INITIALLY DEFERRED
    FOR EACH ROW EXECUTE FUNCTION ledger.assert_entry_balanced();

-- Inside the trigger function: balance is required INDEPENDENTLY for
-- cash and for each security, because a share deficit cannot be settled
-- with dollars inside one entry.
SELECT p.asset_class, p.instrument_id, sum(p.amount) AS imbalance
  INTO v_bad
  FROM ledger.posting p
 WHERE p.entry_id = v_entry_id
 GROUP BY p.asset_class, p.instrument_id
HAVING sum(p.amount) <> 0
 LIMIT 1;

IF FOUND THEN
    RAISE EXCEPTION 'ledger entry % does not balance: % (instrument %) is off by %',
        v_entry_id, v_bad.asset_class,
        COALESCE(v_bad.instrument_id::text,'-'), v_bad.imbalance
        USING ERRCODE = '23514';
END IF;
```

8.3 Views and materialized views

Twelve views and two materialized views present the read models the application consumes, so that no business logic has to be restated in the API layer. The materialized views carry aggregations too expensive to recompute per request and are refreshed CONCURRENTLY by the end-of-day procedure, which requires each to carry a unique index.

Listing 8.4 Views and materialized views, with the security mode of each

```
-- Views and materialized views, with their security mode
       object_name       |  kind   | runs_as
-------------------------+---------+---------
 mv_instrument_daily     | matview | definer
 mv_strategy_leaderboard | matview | definer
 quote_enriched          | view    | definer
 v_account_summary       | view    | invoker
 v_alert_queue           | view    | invoker
 v_blotter               | view    | invoker
 v_book_depth            | view    | definer
 v_fx_board              | view    | definer
 v_open_orders           | view    | invoker
 v_portfolio             | view    | invoker
 v_watchlist             | view    | invoker
(11 rows)
```

The runs_as column records a correction made during development and is worth stating plainly, because it is the kind of defect that produces no error. A view executes with the privileges of its OWNER, and a table owner is exempt from Row Level Security on its own tables. Every view over user data was therefore returning every user’s rows while the policies beneath it appeared correct. The fix was to declare those views WITH (security_invoker = true) so that they execute as the caller, and to add ops.assert_rls_intact(), which fails if a view over a policy-protected table is ever created without it. The five views that remain definer-rights expose only reference and market data, which is not user-specific.

8.4 Transaction control

Every order submission is already a transaction: engine.submit_order either persists the order, its fills, its journal entry and its position change, or none of them. SAVEPOINT allows a finer grain, undoing one statement while the rest of the transaction survives.

Listing 8.5 SAVEPOINT, ROLLBACK TO SAVEPOINT and an outer ROLLBACK

```
-- SAVEPOINT: one statement is undone, the rest of the
--      transaction survives and commits.
 watchlist_id |   name
--------------+----------
          370 | Report A
(1 row)

 watchlist_id |   name
--------------+----------
          371 | Report B
(1 row)

-- Rolled back to the savepoint: only Report A remains pending.
   name
----------
 Report A
(1 row)

-- Outer ROLLBACK: the demonstration leaves nothing behind.
 report_rows_left
------------------
                0
(1 row)
```

Table 8.2 Transaction-control statements and their role

Statement	Effect in this system
BEGIN	Opens the unit of work; the deferred ledger constraint is now pending
SAVEPOINT name	Marks a point the transaction can be returned to
ROLLBACK TO SAVEPOINT	Undoes statements after the mark, keeping the transaction open
COMMIT	Runs every deferred constraint, then makes the work durable
ROLLBACK	Discards the whole unit of work, leaving no trace

8.5 Row Level Security

Authorisation is enforced by the database, not by the API. Policies on the user-facing tables restrict every row to the account that owns it, and the session identity is carried in a session variable that the application sets after authenticating. The demonstration below connects as apex_app, the unprivileged role the API actually uses, and runs one identical query under two identities.

Listing 8.6 The same query, two sessions, two answers

```
-- Row Level Security. Connected as apex_app, the unprivileged
--      role the API uses: the SAME query, two sessions, two answers.
 acting_as_samrender
---------------------
```

```
 342
(1 row)

 symbol |  qty   |  value
--------+--------+----------
 V      | 171.00 | 51057.18
 NFLX   |  48.00 | 33365.28
 TSLA   |  58.00 | 32557.14
 INTC   | 684.00 | 30831.85
(4 rows)

 acting_as_vartika
-------------------
 343
(1 row)

 symbol |  qty   |  value
--------+--------+----------
 SPY    | 87.00 | 53696.40
 QQQ    | 53.00 | 35671.65
 MSFT   | 30.00 | 22536.60
 JPM    | 71.00 | 17750.00
(4 rows)

-- Neither session can widen its own view: the policy sits on the
-- base table, and the view runs with security_invoker = true.
 accounts_visible
------------------
                1
(1 row)
```

The final count is the point: a session can see exactly one account, and no query it can write will widen that, because the restriction is attached to the table rather than to the statement. A compliance officer sees everything, and that escape hatch reads the role from identity.app_user rather than from the client, so a client cannot claim to be compliance.

Listing 8.7 A representative policy

```
CREATE POLICY p_position_own ON trading.position FOR SELECT
    USING (identity.current_user_is_privileged()
        OR account_id IN (SELECT a.account_id FROM trading.account a
                          WHERE a.user_id = identity.current_user_id()));
```

8.6 Partitioning and the index strategy

market.tick, market.candle_1m and trading.execution are declaratively range-partitioned by month, and ops.ensure_time_partitions keeps a runway of future partitions ahead of the data. Partitioning buys two things here: the planner prunes whole months at plan time, as measured in Section 7.10, and an expired month can be detached in constant time rather than deleted row by row.

Listing 8.8 Index access methods in use, counted from pg_index

```
-- The index strategy, read back from the catalogue
 method |  n  |                          examples
--------+-----+-------------------------------------------------------------
 btree  | 209 | account_account_no_key, account_pkey, account_snapshot_p
 brin   |   4 | ix_audit_time_brin, ix_candle_1m_bucket_brin, ix_exec_ti
 gin    |   4 | ix_alert_evidence, ix_instrument_metadata, ix_instrument
 gist   |   1 | no_overlapping_halts
(4 rows)
```

Table 8.3 Index choices and the reason for each

Index	Type	Why this type
ix_book_bids / ix_book_asks	Partial B-tree	Covers only resting orders; serves the matching ORDER BY with no sort
ix_candle_1m_bucket_brin	BRIN	Append-ordered time column; a fraction of a B-tree's size
ix_instrument_symbol_trgm	GIN (trigram)	Substring and fuzzy symbol search for the search box
ix_alert_evidence	GIN (jsonb_path_ops)	Containment queries into the surveillance evidence document
no_overlapping_halts	GiST (btree_gist)	EXCLUDE constraint: two halts on one instrument cannot overlap
ix_alert_open	Partial B-tree	Indexes open alerts only, which is the only set ever queried

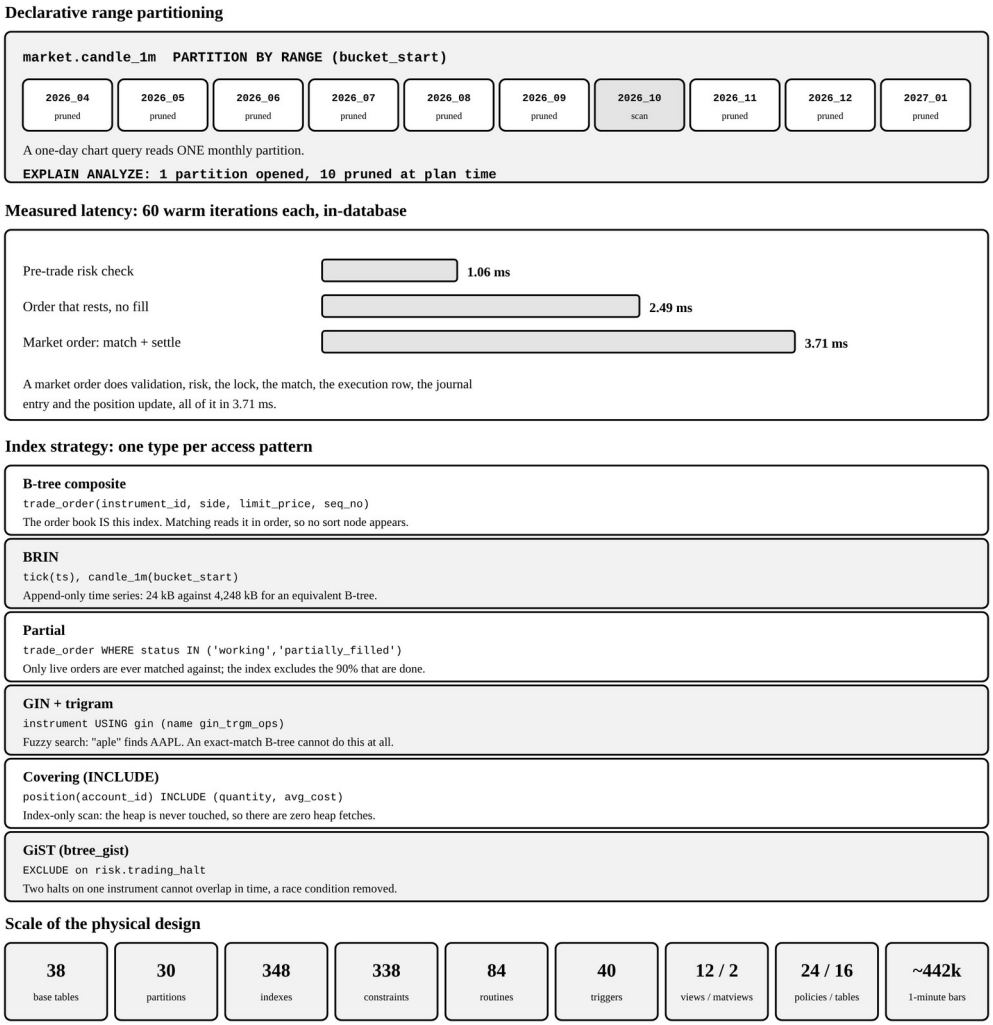


Figure 8.2 Physical design: partitioning, index strategy and measured latency

8.7 The audit trail

audit.activity_log records the before and after image of every material change to an order, account, user or alert, together with the acting user, the database role, the client address and the transaction id. It is written by trigger, so it cannot be bypassed by writing to the table directly, and it is append-only: a second trigger rejects UPDATE and DELETE on the log itself.

Listing 8.9 The audit trail, and an attempt to rewrite it

```
-- The append-only audit trail, written by a trigger
 log_id | table_name  | operation |  pk  | acted_by |      acted_at
--------+-------------+-----------+------+----------+---------------------
  15896 | app_user    | UPDATE    | 348  |          | 2026-10-18 23:09:55
  15895 | trade_order | UPDATE    | 9718 |      342 | 2026-10-18 23:09:12
  15894 | trade_order | INSERT    | 9718 |      342 | 2026-10-18 23:09:12
  15893 | app_user    | UPDATE    | 342  |          | 2026-10-18 23:09:03
  15859 | trade_order | UPDATE    | 9702 |      342 | 2026-10-18 22:58:10
  15858 | trade_order | INSERT    | 9702 |      342 | 2026-10-18 22:58:10
(6 rows)

-- The log cannot be rewritten, not even by the schema owner:
psql:q17_audit.sql:7: ERROR:  audit.activity_log is append-only (attempted UPDATE)
CONTEXT:  PL/pgSQL function audit.forbid_mutation() line 3 at RAISE
```

8.8 Concurrency control

An order book is a contended resource: many sessions attempt to trade the same instrument at the same moment, and the failure mode is the double fill, in which two buyers each read the same resting sell order and both match against it. The engine serialises matching per instrument with a transaction-scoped advisory lock, so different instruments proceed in parallel while one instrument is matched one order at a time.

Listing 8.10 The per-instrument serialisation point

```
-- One book at a time, and only for the life of this transaction.
-- Different instruments never contend with each other.
PERFORM pg_advisory_xact_lock(hashtext('apex_book'), p_instrument_id);
```

The claim is tested rather than asserted. db/test/concurrency.sh rests a single sell order for three thousand shares and then launches twelve independent psql sessions, each submitting twenty-five buy orders against it, so that total demand exactly equals available supply. Afterwards four things must hold: the resting order is filled exactly once over, the ledger still balances, no account is overdrawn, and the number of shares bought equals the number sold.

Listing 8.11 Twelve concurrent sessions, three hundred fills, no double fill

```
=== setup: fresh fixture, one large resting sell order ===
  resting sell order #8803 for 3000 @ 182.00
  launching 12 concurrent buyers x 25 orders each
  (total demand 3000 shares against 3000 available)
  completed in 7s

=== invariant checks ===
```

```
-[ RECORD 1 ]---------------+--------------
resting_quantity            | 3000.00000000
resting_filled              | 3000.00000000
resting_status              | filled
total_fills                 | 300
total_shares_traded         | 3000.00000000
overfilled_orders           | 0
rejected_orders             | 0
shares_created_or_destroyed | 0.00000000
cash_created_or_destroyed   | 0.00000000

           check_name            | passed |             detail
---------------------------------+--------+--------------------------------
 balance_cache_matches_journal   | t      | 0 drifted slot(s)
 every_entry_balances            | t      | 0 unbalanced entry/entries
 closed_system_conservation      | t      | 0 non-conserved asset(s)
 no_unauthorised_overdraft       | t      | 0 overdrawn account(s)
 every_cash_posting_has_currency | t      | 0 posting(s) without a currency
(5 rows)

CONCURRENCY TEST PASSED -- 12 sessions, no double-fill, ledger exact
```

9. PROJECT OUTPUT SCREENSHOTS

The guidelines ask this section to show four things: the structure of the database, sample records, the execution of important queries, and the application. Each is below. The transcripts are verbatim psql output; the screenshots were captured from the running application against the seeded demonstration database, in its own dark theme, and are reproduced in full colour. Every number visible in them was computed by the database.

9.1 Database structure

Thirty-eight base tables across eleven data-bearing schemas, listed as the catalogue holds them. The twelfth schema, engine, contains functions only.

Listing 9.1 Every base table in the database, grouped by schema

```
-- Every base table in the database, by schema
  schema   |                                        tables
-----------+-------------------------------------------------------------------------------
 analytics | account_snapshot
 audit     | activity_log
 identity  | app_user, login_attempt, user_session
 ledger    | balance, journal_entry, posting
 market    | candle_1m, quote, tick
 ops       | config, job_run, schema_migration
 reference | corporate_action, currency, exchange, fx_rate, instrument, trading_session
 risk      | alert_fire, margin_call, price_alert, risk_limit, trading_halt
 social    | copy_link, copy_queue, copy_subscription, follow, strategy, watchlist,
     watchlist_item
 surveil   | alert, rule
 trading   | account, execution, position, trade_order
(11 rows)
```

A single table, described the way psql describes one, shows what the design puts around each relation: a composite primary key, a partial index, a CHECK constraint, two foreign keys and a Row Level Security policy.

Listing 9.2 \d trading.position: columns, indexes, constraints and its policy

```
-- The structure of one table, as psql describes it
                      Table "trading.position"
    Column     |           Type           | Collation | Nullable | Default
---------------+--------------------------+-----------+----------+---------
 account_id    | bigint                   |           | not null |
 instrument_id | integer                  |           | not null |
 quantity      | money_amt                |           | not null | 0
 avg_cost      | numeric(20,8)            |           | not null | 0
 realized_pnl  | money_amt                |           | not null | 0
 opened_at     | timestamp with time zone |           | not null | now()
 updated_at    | timestamp with time zone |           | not null | now()
Indexes:
    "position_pkey" PRIMARY KEY, btree (account_id, instrument_id)
    "ix_position_instrument" btree (instrument_id) WHERE quantity::numeric <> 0::numeric
Check constraints:
    "position_avg_cost_check" CHECK (avg_cost >= 0::numeric)
Foreign-key constraints:
    "position_account_id_fkey" FOREIGN KEY (account_id) REFERENCES
       trading.account(account_id) ON DELETE CASCADE
    "position_instrument_id_fkey" FOREIGN KEY (instrument_id) REFERENCES
       reference.instrument(instrument_id)
Policies:
    POLICY "p_position_own" FOR SELECT
      USING ((identity.current_user_is_privileged() OR (account_id IN ( SELECT
          a.account_id
```

```
   FROM trading.account a
  WHERE (a.user_id = identity.current_user_id())))))
```

9.2 Sample records

The demonstration data holds eight users, eight trading accounts and five system accounts. The accounts are deliberately varied: two margin accounts in US dollars, one in pounds sterling, and cash accounts denominated in rupees, euros and dirhams, so that the multi-currency ledger is exercised rather than merely present.

Listing 9.3 Sample records from identity.app_user and trading.account

```
-- Sample records, read straight from the seeded tables
-- identity.app_user
 user_id |       email        |       full_name       |    role    | kyc_state
---------+--------------------+-----------------------+------------+-----------
     342 | samrender@apex.io  | Samrender Singh Saini | trader     | verified
     343 | vartika@apex.io    | Vartika Chaudhary     | trader     | verified
     344 | paritosh@apex.io   | Kumar Paritosh        | trader     | verified
     345 | sam@apex.io        | Sam Saini             | trader     | verified
     346 | ginni@apex.io      | Ginni Kaur            | trader     | verified
     347 | pari@apex.io       | Pari Sharma           | trader     | verified
     348 | compliance@apex.io | Ravi Compliance       | compliance | verified
     349 | admin@apex.io      | Apex Admin            | admin      | verified
(8 rows)

-- trading.account
 account_id | account_no | user_id |  kind  | base_currency | allow_short_selling
------------+------------+---------+--------+---------------+---------------------
        375 | APX-000342 |     342 | margin | USD           | f
        376 | APX-000343 |     343 | margin | USD           | f
        377 | APX-000344 |     344 | margin | GBP           | f
        378 | APX-000345 |     345 | cash   | INR           | f
        379 | APX-000346 |     346 | cash   | EUR           | f
        380 | APX-000347 |     347 | cash   | AED           | f
        381 | APX-000348 |     348 | cash   | USD           | f
        382 | APX-000349 |     349 | cash   | USD           | f
(8 rows)
```

Table 9.1 gives the login credentials for the demonstration, which are the same on any freshly seeded database.

Table 9.1 Demonstration logins. The password is Apex@2026 for every account.

Login	Name	Account	Base currency	Role
samrender@apex.io	Samrender Singh Saini	APX-000342	USD (margin)	trader
vartika@apex.io	Vartika Chaudhary	APX-000343	USD (margin)	trader
paritosh@apex.io	Kumar Paritosh	APX-000344	GBP (margin)	trader
sam@apex.io	Sam Saini	APX-000345	INR (cash)	trader
ginni@apex.io	Ginni Kaur	APX-000346	EUR (cash)	trader
pari@apex.io	Pari Sharma	APX-000347	AED (cash)	trader
compliance@apex.io	Ravi Compliance	APX-000348	USD (cash)	compliance
admin@apex.io	Apex Admin	APX-000349	USD (cash)	admin

9.3 Execution of important queries

Section 7 reproduces eleven query transcripts in full and Section 8 a further eleven, each with the statement that produced it. Rather than repeat them, this section records the one operation that exercises the whole system at once: the pre-trade gate refusing three orders. It is the clearest single demonstration that the rules are in the database, because the refusals are produced by risk.pretrade_check and not by any code in the application.

Listing 9.4 Three orders refused before anything reaches the order book

```
-- Pre-trade risk: three orders the database refuses to accept.
--      Trailing zeros are stripped from the messages for width.
-- (a) Selling stock the account does not hold, short selling off
  status  |                                  reject_reason
----------+-------------------------------------------------------------------------------
 rejected | cannot sell 5 NVDA: you hold 0. Short selling is not enabled on this account.
(1 row)

-- (b) A buy far beyond the single-order notional cap
  status  |                       reject_reason
----------+-------------------------------------------------------------
 rejected | order value 20000000 USD exceeds your limit of 1000000 USD
(1 row)

-- (c) A limit price outside the instrument price band
  status  |                 reject_reason
----------+-----------------------------------------------
 rejected | price 20 outside 20% band [136.8080, 205.2120]
(1 row)
```

9.4 Application screenshots

Ten screens follow. Three are reproduced here at their own size; the six wide views are placed on the sideways pages that follow this section, where their text stays legible in print.

9.4.1 Authentication

Six named demonstration accounts are seeded, one per team member and two additional profiles, together with a compliance officer and an administrator. Passwords are never stored: identity.hash_password uses pgcrypto with a per-row salt, and identity.authenticate compares the hash, increments a failure counter and applies a lockout, all inside the database.

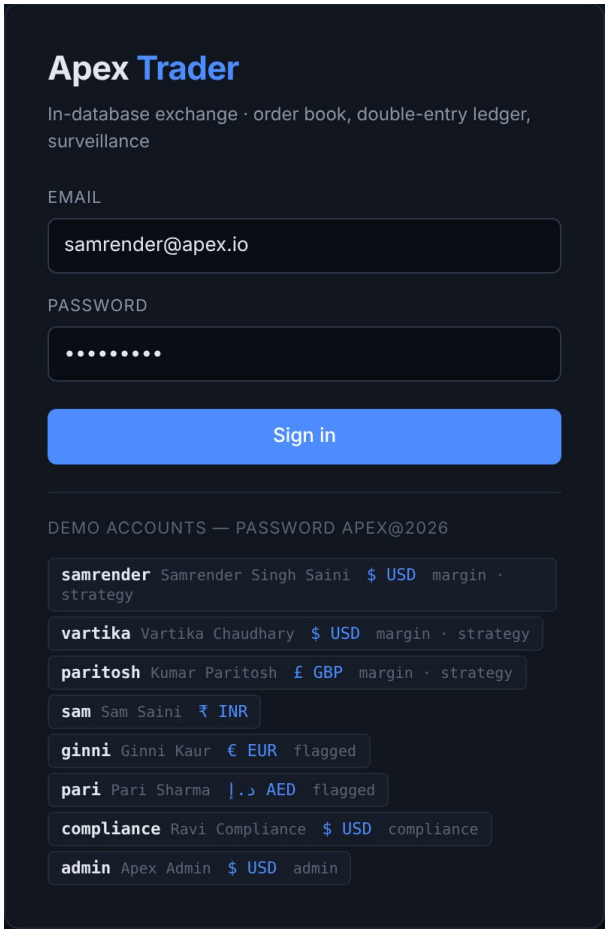


Figure 9.1 Sign-in. Credentials are verified by identity.authenticate(), not by the API

9.4.2 The order ticket

The ticket shows the estimated value of the order and states plainly whether it will cross the spread. The line beneath it carries buying power, cash and net asset value, each read from the ledger rather than from a cached field, and notes whether short selling is enabled on the account.

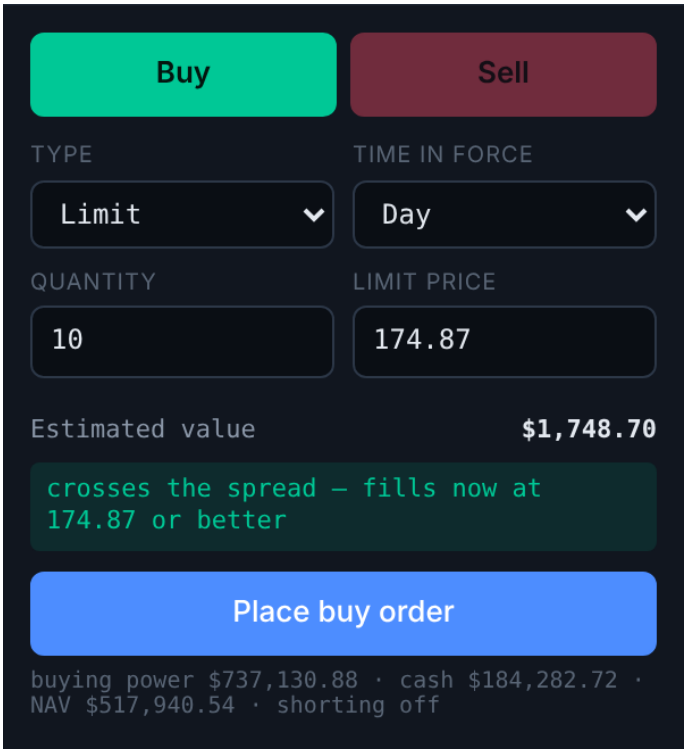


Figure 9.2 The order ticket, with pre-trade figures taken from the ledger

9.4.3 Symbol search

Search matches on substrings and tolerates misspelling because the index is a GIN trigram index rather than a B-tree; a prefix index could not answer a query for the middle of a company name. A symbol that is not yet in the security master can be added on demand, which inserts the instrument and backfills its history.

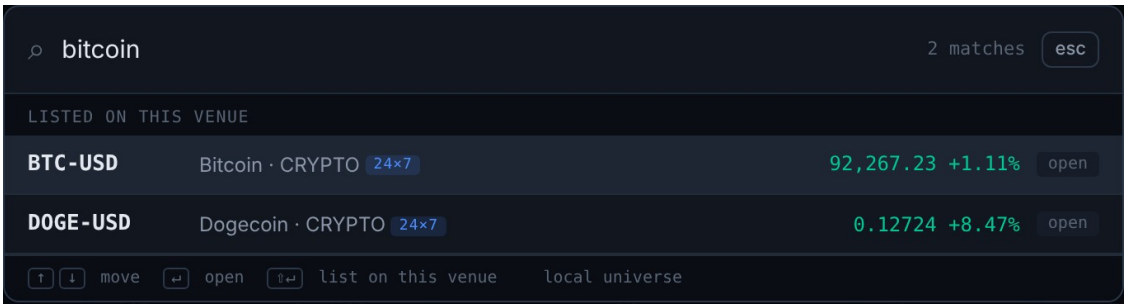


Figure 9.3 The search palette, served by a GIN trigram index over symbol and name

9.4.4 The remaining screens

Figure 9.4 is the trading terminal: a candlestick chart with moving averages, Bollinger bands, volume and RSI, a live depth-of-market ladder, and the order ticket. The indicators are not computed in the browser; they are returned by market.candles_with_indicators, which derives every one of them with window functions over the minute-bar table.

Figures 9.5 and 9.6 are the clearest statement of the design. The positions tab shows what the account holds; the ledger tab shows every journal entry that produced those holdings. The first is derived from the second, and Section 7.11 includes the invariant that proves it. Entry #5819 in Figure 9.6 is a complete double entry: one share of AAPL into the security account, and 175.0449 dollars out of the cash account. They are two legs of one entry, and they sum to zero per asset class.

Figure 9.7 reports cumulative return, maximum drawdown, Sharpe ratio and win rate, all four returned by analytics.performance_stats from the daily NAV snapshot series in one statement. Figure 9.8 is the same account read the other way round: not what it is worth now, but every order, fill and cash movement that brought it there.

Figure 9.9 lists equities, ETFs and crypto together. Crypto instruments carry trades_24x7, so reference.is_instrument_open returns true for them at all hours while an equity is closed outside its exchange session; the distinction is reference data, not a conditional in the application.

Figure 9.10 is the compliance console, the view that makes the database’s own guarantees visible. It lists the eighteen structural invariants with their current status, the open surveillance alerts with the evidence that produced each, and the operational actions. It is reachable only by a user whose role in identity.app_user is compliance or admin. The critical alert shown is a wash trade: the scanner found six executions in which the same beneficial owner stood on both sides, totalling 1,200 shares and 54,096 dollars of notional.



Figure 9.4 The trading terminal: chart, depth of market, and the order ticket

Open orders | Positions 19 | Fills | Time & sales | Ledger | ★ Watchlist 1 | 🔔 Alerts | ⭳

SYMBOL	NAME	QTY	AVG COST	MARK	VALUE	UNREALISED	%	REALISED	% OF BOOK
V	Visa Inc.	171	281.71	298.58	$51,057.18	$2,883.98	+5.99%	-$39.86	15.3%
NFLX	Netflix Inc.	48	661.15	695.11	$33,365.28	$1,630.26	+5.14%	$0.00	10.0%
TSLA	Tesla Inc.	58	383.71	561.33	$32,557.14	$10,302.07	+46.29%	$692.41	9.8%
INTC	Intel Corporation	684	44.77	45.08	$30,831.85	$206.20	+0.67%	$692.12	9.2%
SPY	SPDR S&P 500 ETF Trust	44	578.66	617.20	$27,156.80	$1,695.95	+6.66%	$126.72	8.1%
NVDA	NVIDIA Corporation	305	98.26	70.17	$21,402.06	-$8,566.66	-28.59%	$0.00	6.4%
QQQ	Invesco QQQ Trust	31	590.80	673.05	$20,864.55	$2,549.75	+13.92%	$51.54	6.3%
AAPL	Apple Inc.	112	195.80	174.87	$19,585.44	-$2,344.21	-10.69%	$0.00	5.9%
SOL-USD	Solana	62.26	192.75	313.62	$19,524.98	$7,524.98	+62.71%	$0.00	5.9%
GOOGL	Alphabet Inc.	154	142.60	117.15	$18,041.10	-$3,919.58	-17.85%	$0.00	5.4%

Figure 9.5 Open positions, with unrealised profit against the live mark

Open orders | Positions 19 | Fills | Time & sales | Ledger | ★ Watchlist 1 | 🔔 Alerts | ⭳

WHEN	ENTRY	KIND	ASSET	SYMBOL	AMOUNT	DESCRIPTION
Oct 18 23:09:12	#5826	trade_settlement	security	BTC-USD	0.0015	0.00150000 x BTC-USD @ 92267.23000000 USD
Oct 18 23:09:12	#5826	trade_settlement	cash	—	-138.5392	0.00150000 x BTC-USD @ 92267.23000000 USD
Oct 18 22:58:09	#5819	trade_settlement	security	AAPL	1.0000	1.00000000 x AAPL @ 174.87000000 USD
Oct 18 22:58:09	#5819	trade_settlement	cash	—	-175.0449	1.00000000 x AAPL @ 174.87000000 USD
Oct 18 22:58:08	#5811	fx_conversion	cash	—	41,783.8000	Manual conversion from the dashboard
Oct 18 22:58:08	#5811	fx_conversion	cash	—	-500.0000	Manual conversion from the dashboard
Oct 18 22:58:07	#5810	trade_settlement	security	AAPL	10.0000	10.00000000 x AAPL @ 174.87000000 USD
Oct 18 22:58:07	#5810	trade_settlement	cash	—	-1,750.4487	10.00000000 x AAPL @ 174.87000000 USD
Oct 15 19:19:13	#5794	trade_settlement	security	JPM	8.0000	backfill 8.00000000 @ 237.21000000 USD
Oct 15 19:19:13	#5794	trade_settlement	cash	—	-1,899.5777	backfill 8.00000000 @ 237.21000000 USD

Figure 9.6 The ledger: each trade settlement is a pair of balancing legs

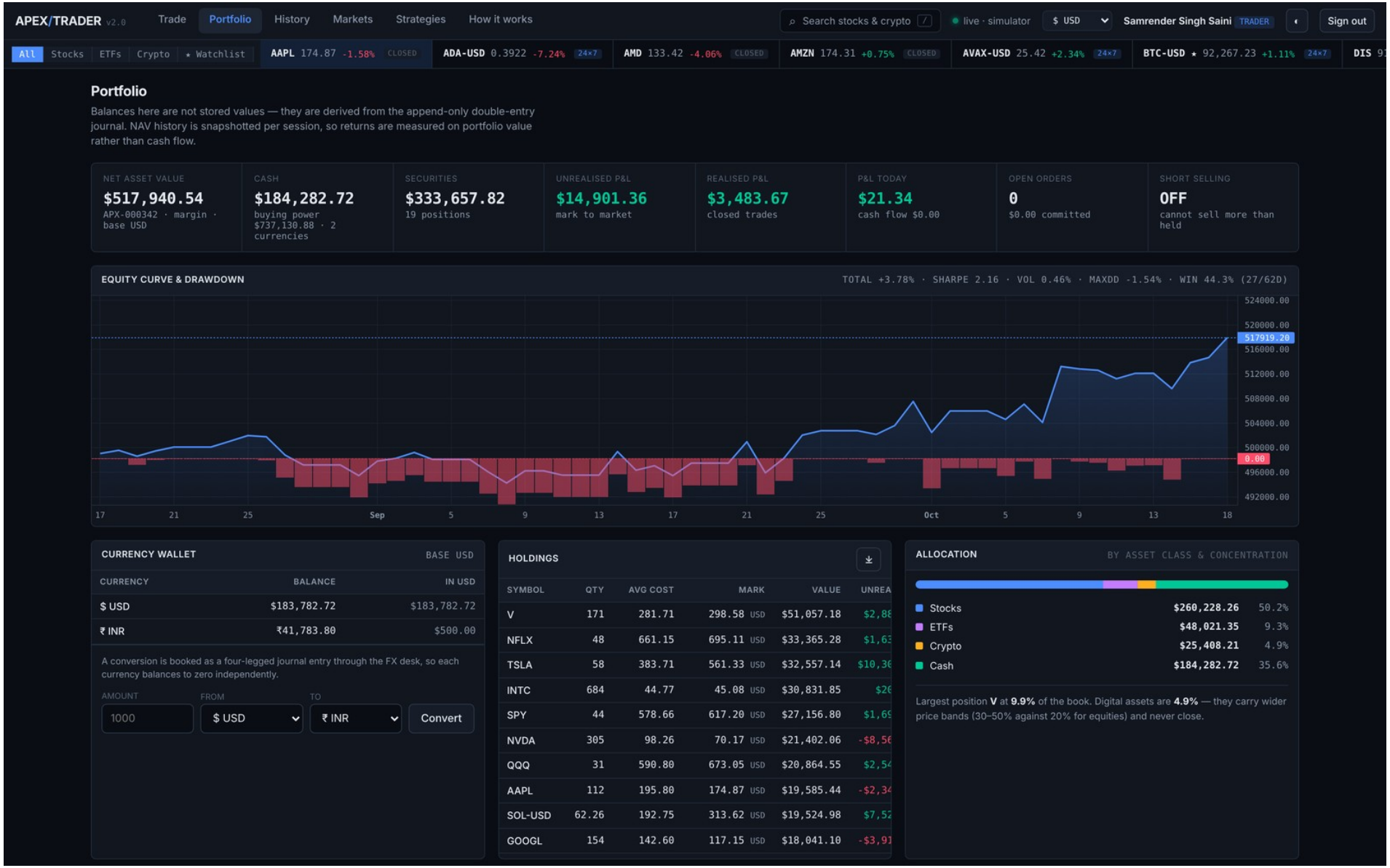


Figure 9.7 Portfolio: net asset value curve, allocation and performance statistics

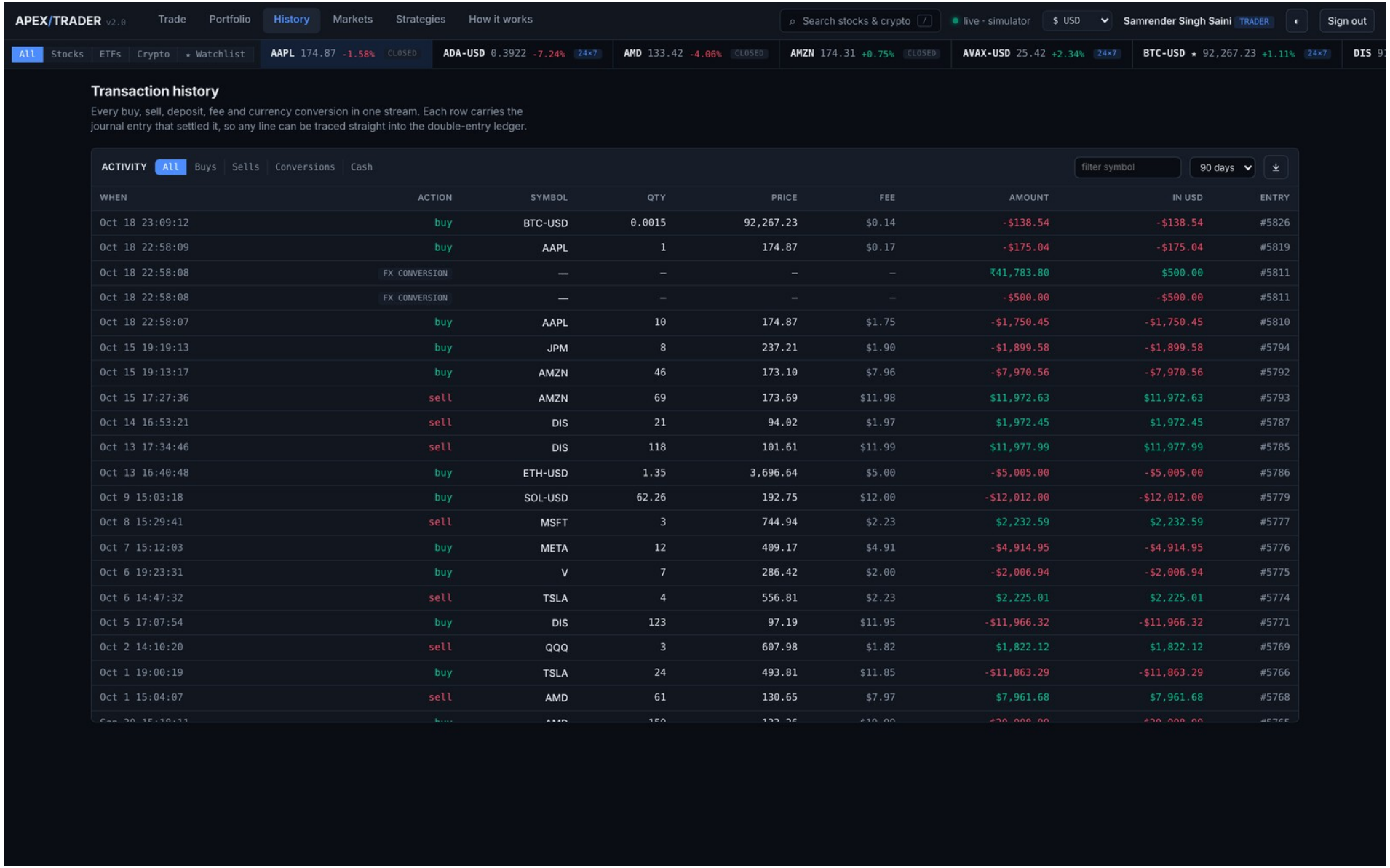


Figure 9.8 History: the order blotter, fills and the cash statement

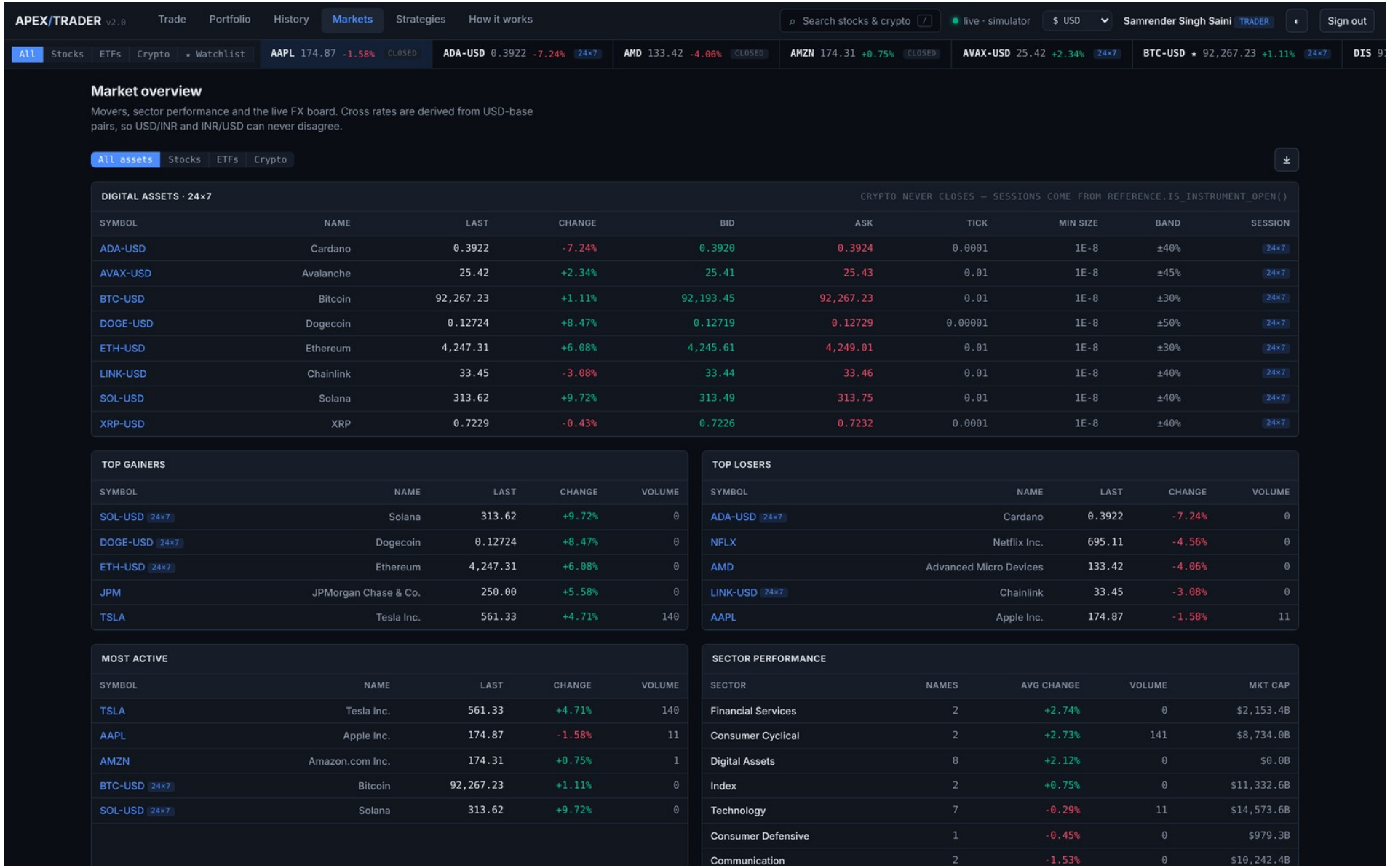


Figure 9.9 Markets: equities, ETFs and crypto, with session state per instrument

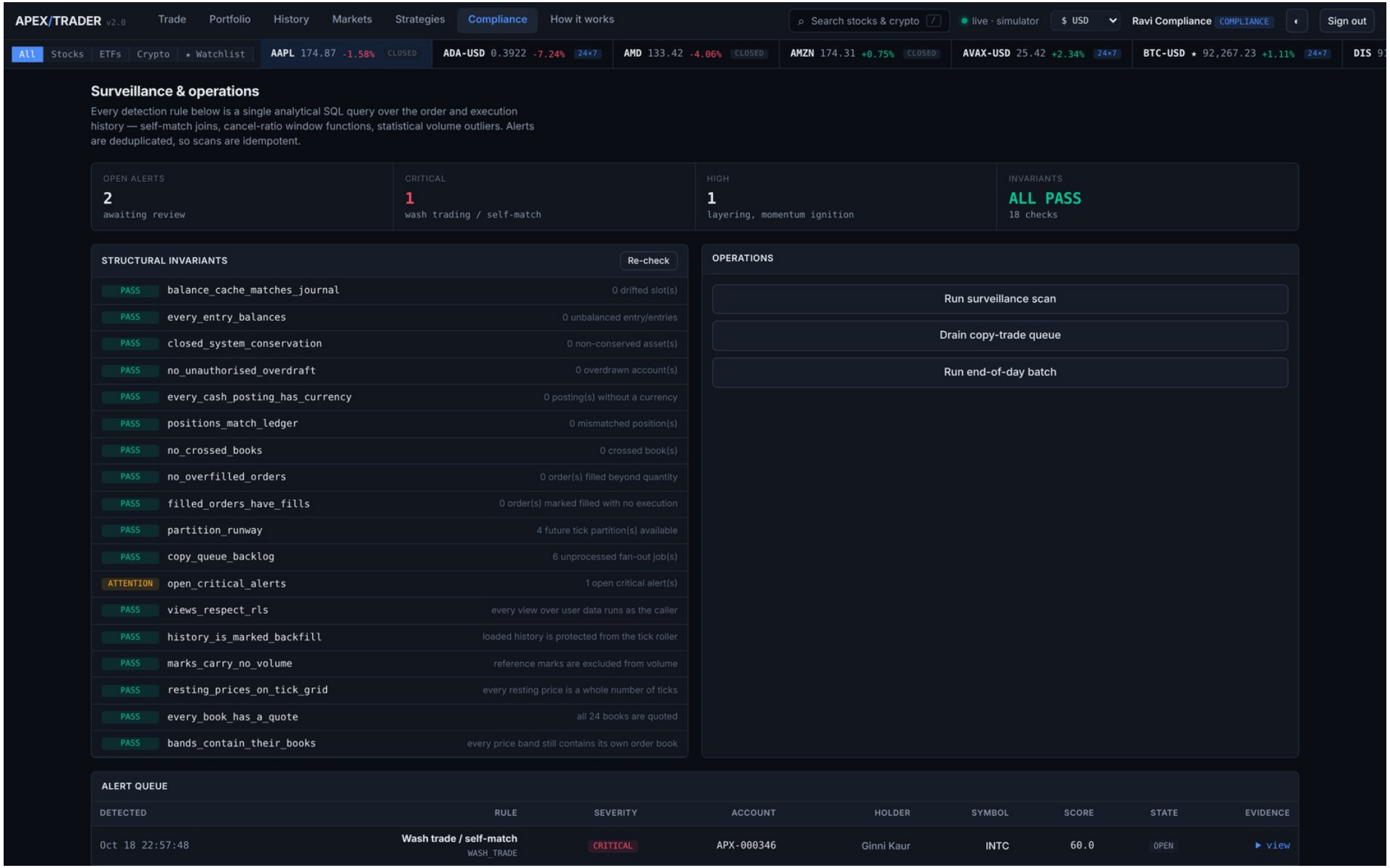


Figure 9.10 Compliance: invariants, the alert queue, and operational actions

9.5 Verification

Four independent suites are run before any submission of this project. They check different things on purpose: the SQL suite checks the rules, the concurrency suite checks them under contention, the API suite checks the contract, and the browser suite checks that the screens above actually render what the database returned.

Table 9.2 Verification suites and their results

Suite	What it exercises	Result
db/test (5 files)	Ledger, engine, risk, currency and crypto rules	150 assertions passed
db/test/concurrency.sh	12 sessions x 25 orders against one resting order	Passed, no double fill
scripts/api_smoke.py	Every REST path, authorisation and error case	118 checks passed
scripts/ui_check.py	Browser rendering of all seven screens	51 checks passed
ops.health_check()	Eighteen structural invariants on live data	18 of 18 reporting

Listing 9.5 The SQL assertion suite

```
COMMIT

  suite   | passed | failed
----------+--------+--------
 crypto   |     31 |      0
 currency |     32 |      0
 engine   |     32 |      0
 ledger   |     16 |      0
 risk     |     39 |      0
(5 rows)

all 150 assertions passed
```

The suite itself once had a defect. For a period it reported every assertion green while silently checking fewer of them: a type mismatch inside one helper raised an error that psql swallowed because ON_ERROR_STOP was not set. The cast was corrected and scripts/test.sh now fails on any error psql reports, not only on a failed assertion.

10. CONCLUSION

10.1 Summary of the completed project

Apex Trader is a complete, working trading system in which the database is not a store behind the application but the application itself. Thirty-eight base tables across twelve bounded-context schemas hold the data; eighty-three functions and one procedure hold the behaviour; fifteen triggers, five hundred and forty NOT NULL constraints, seventy-one CHECK constraints, fifty-seven foreign keys, an exclusion constraint and a deferred constraint trigger hold the rules. The service tier above it validates request shape and shapes JSON, and contains no business logic that could disagree with the database.

Every objective set out in Section 3 was met. The matching engine executes inside the database with price-time priority and is proved correct under twelve-way concurrency. The ledger is double entry, its balances are derived rather than stored, and its zero-sum property is enforced by a constraint rather than by convention. Authorisation is enforced by Row Level Security, which an incorrect query cannot bypass. Seven currencies, two asset classes and six order types are supported. The system checks eighteen structural invariants on its own live data and exposes the result through its health endpoint and its compliance console.

10.2 Main features and benefits

Table 10.1 Principal features and what each is for

Feature	Benefit it delivers
In-database matching with price-time priority	The order book is correct under concurrency because the database, not the application, serialises access to it
Double-entry ledger with derived balances	A balance cannot drift from its history, because the history is the balance
Deferred zero-sum constraint	Money can neither be created nor destroyed, and the guarantee survives any future code that writes to the ledger
Row Level Security with security_invoker views	A forgotten predicate returns nothing rather than another trader’s rows
Declarative partitioning and a deliberate index strategy	The chart query opens one month of 442,000 rows and returns in well under a millisecond
Append-only audit trail written by trigger	Every material change is reconstructable, and the record cannot be edited
Eighteen invariants checked on live data	Defects that produce no error message are still detected

Feature	Benefit it delivers
Checksummed, ordered migrations	The examiner, the developers and the demonstration machine run the same schema

10.3 What was learnt

Three findings during development changed how the system is built, and each shared a shape: the defect produced no error, only a wrong answer.

The first was that every view in the database was bypassing Row Level Security. A view runs with the privileges of its owner, and a table owner is exempt from policies on its own tables, so the policies were correct and the data was exposed anyway. The repair was security_invoker; the lasting change was ops.assert_rls_intact(), which fails if the mistake is ever made again.

The second was that the candle roller was destroying backfilled history, described in Section 6. 7. The third was that the test suite reported green while checking less than it claimed, described in Section 9.7. In all three cases the system was corrected, and in all three cases a check was added so that the correctness becomes a property the database asserts rather than a property the authors remember.

10.4 Future enhancement

Table 10.2 Planned enhancements

Enhancement	Approach
Logical replication to a read replica	Route analytics and chart reads to a replica; the write path is already a single function call
Timescale-style continuous aggregates	Replace the scheduled candle rollup with an incrementally maintained aggregate
Options and futures	Reference data already models tick size, lot size and contract metadata; the ledger needs a third asset class
Settlement cycles (T+1)	The journal already carries occurred_at; a settlement date and a pending-settlement account complete it
Order book depth beyond top of book	The resting-order index already supports it; only aggregation and transport are missing
Partition detachment to cold storage	ops.detach_old_partitions exists; the remaining work is archival and restore
Formal isolation testing with pgbench	Extend the concurrency proof to serialisation anomalies under REPEATABLE READ

11. REFERENCES

[1] Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, 7th edition, Pearson, 2016.

[2] Silberschatz, A., Korth, H. F. and Sudarshan, S., Database System Concepts, 7th edition, McGraw-Hill, 2019.

[3] Date, C. J., An Introduction to Database Systems, 8th edition, Addison-Wesley, 2003.

[4] Codd, E. F., "A Relational Model of Data for Large Shared Data Banks", Communications of the ACM, vol. 13, no. 6, pp. 377-387, 1970.

[5] Codd, E. F., "Recent Investigations into Relational Data Base Systems", Proceedings of IFIP Congress, 1974. (Boyce-Codd Normal Form)

[6] Kleppmann, M., Designing Data-Intensive Applications, O'Reilly Media, 2017.

[7] The PostgreSQL Global Development Group, PostgreSQL 16 Documentation, 2024. Available: https://www.postgresql.org/docs/16/

[8] PostgreSQL Documentation, "Row Security Policies". Available: https://www.postgresql.org/docs/16/ddl-rowsecurity.html

[9] PostgreSQL Documentation, "Table Partitioning". Available: https://www.postgresql.org/docs/16/ddl-partitioning.html

[10] PostgreSQL Documentation, "CREATE TRIGGER" and "SET CONSTRAINTS". Available: https://www.postgresql.org/docs/16/sql-createtrigger.html

[11] PostgreSQL Documentation, "Explicit Locking: Advisory Locks". Available: https://www.postgresql.org/docs/16/explicit-locking.html

[12] PostgreSQL Documentation, "BRIN Indexes" and "GIN Indexes". Available: https://www.postgresql.org/docs/16/brin-intro.html

[13] PostgreSQL Documentation, "pgcrypto". Available: https://www.postgresql.org/docs/16/pgcrypto.html

[14] Harris, L., Trading and Exchanges: Market Microstructure for Practitioners, Oxford University Press, 2003.

[15] Johnson, B., Algorithmic Trading and DMA: An Introduction to Direct Access Trading Strategies, 4Myeloma Press, 2010.

[16] International Organization for Standardization, ISO 4217: Codes for the representation of currencies, 2015.

[17] International Organization for Standardization, ISO 6166: Securities — International Securities Identification Numbering system (ISIN), 2021.

[18] FIX Trading Community, FIX Protocol Specification, version 4.4. Available: https://www.fixtrading.org/

[19] Securities and Exchange Board of India, Master Circular on Surveillance of Securities Market, 2023.

[20] Ramírez, S., FastAPI Documentation. Available: https://fastapi.tiangolo.com/

[21] MagicStack, asyncpg Documentation. Available: https://magicstack.github.io/asyncpg/

[22] TradingView, Lightweight Charts Documentation. Available: https://tradingview.github.io/lightweight-charts/

[23] Yahoo Finance market data endpoints, accessed October 2026.